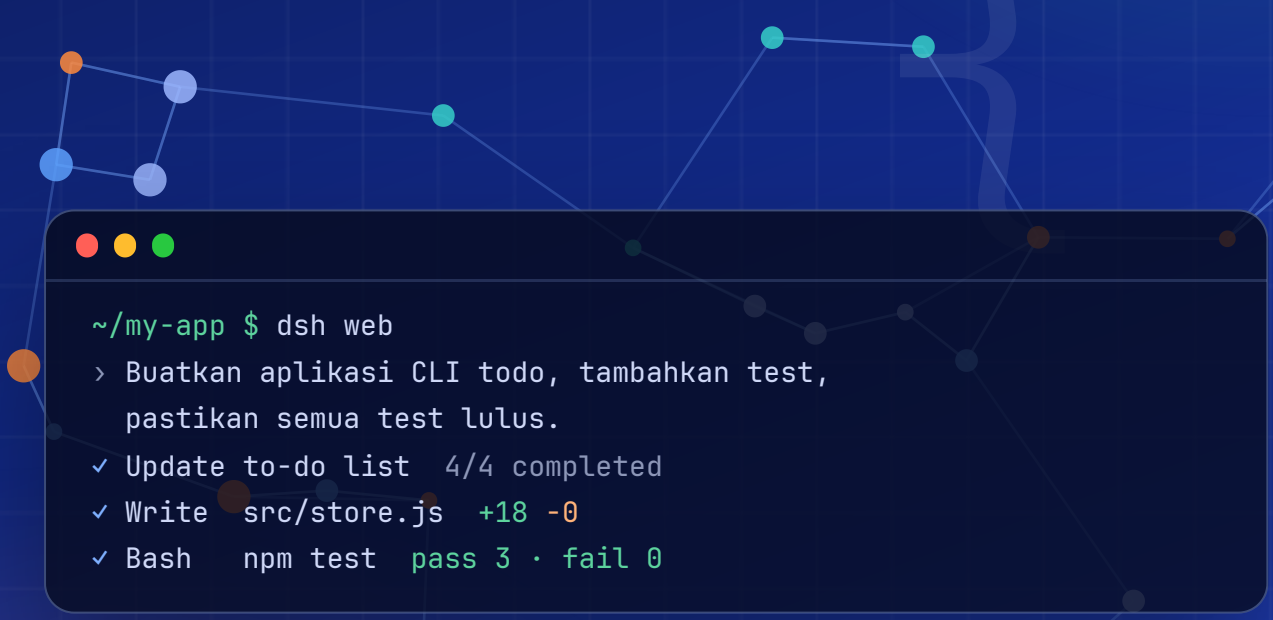
PANDUAN PRAKTIS

Vibe Coding dengan DeepSeek Harness

Agen pemrograman berbasis plugin untuk programmer masa kini — dari instalasi, sesi coding pertama, hingga otomasi dan kustomisasi.



PENULIS
Syamsuddin

Edisi Pertama — Edisi Digital
2026

Vibe Coding dengan DeepSeek Harness

Panduan Praktis Agen Pemrograman Berbasis Plugin untuk Programmer Masa Kini

Syamsuddin

Edisi Pertama — Edisi Digital
2026

Vibe Coding dengan DeepSeek Harness
Panduan Praktis Agen Pemrograman Berbasis Plugin untuk Programmer Masa Kini

Penulis: Syamsuddin
Desain sampul dan tata letak: Syamsuddin

Hak cipta © 2026 Syamsuddin. Hak cipta dilindungi undang-undang. Dilarang memperbanyak sebagian atau seluruh isi buku ini dalam bentuk apa pun tanpa izin tertulis dari pemegang hak cipta, kecuali kutipan singkat untuk keperluan ulasan dan pendidikan.

Edisi Pertama — Edisi Digital, Oktober 2026
ISBN: [diisi setelah terbit dari Perpusnas]
Penerbit: [nama dan alamat penerbit: diisi]

Huruf: Bitstream Charter (isi), Inter (judul dan keterangan), JetBrains Mono (kode).
Ukuran: A4 (21 × 29,7 cm).

DeepSeek dan DeepSeek Harness adalah nama dan karya milik DeepSeek AI. Buku ini adalah panduan independen dan bukan publikasi resmi DeepSeek AI. DeepSeek Harness dirilis dengan lisensi MIT. Screenshot antarmuka diambil penulis dari Web UI yang dibangun dari kode sumber proyek versi 0.2.2-alpha.2; sesi contoh pada Bab 5 bersifat ilustratif. Gambar 4.1 bersumber dari dokumentasi resmi repositori proyek.

Isi buku disusun dengan cermat, tetapi perangkat lunak yang dibahas masih berstatus developer preview. Penulis tidak bertanggung jawab atas kerugian yang timbul dari penggunaan perangkat lunak tersebut; ikuti selalu panduan keselamatan resmi proyek.

Kata Pengantar

Beberapa tahun terakhir, cara kita menulis perangkat lunak berubah lebih cepat daripada kebiasaan kita menyesuaikan diri. Model bahasa kini mampu membaca repositori, menulis berkas, menjalankan test, dan memperbaiki kesalahannya sendiri. Lahirlah gaya kerja yang populer disebut **vibe coding**: kita menjelaskan maksud dengan bahasa sehari-hari, dan AI mengerjakan sebagian besar ketikannya.

Gaya ini menyenangkan, tetapi bukan tanpa jebakan. Agen yang terlalu bebas dapat mengubah berkas yang tidak seharusnya, menjalankan perintah berisiko, atau dengan percaya diri melaporkan pekerjaan yang belum benar-benar selesai. Di sinilah peran sebuah *agent harness* menjadi penting: kerangka yang memberi agen kemampuan sekaligus rambu-rambu.

Buku ini membahas **DeepSeek Harness**, agent harness sumber terbuka dari DeepSeek AI yang dibangun dengan filosofi "semuanya adalah plugin". Penulis menyusunnya setelah membaca dan menganalisis langsung kode sumber serta dokumentasi proyek, lalu membangun dan menjalankan Web UI-nya untuk mengambil setiap screenshot yang Anda lihat. Dengan begitu, panduan ini berpijak pada perilaku perangkat lunak yang sebenarnya, bukan sekadar terjemahan brosur.

Sasaran pembaca buku ini adalah programmer — pemula yang baru mencicipi asisten AI maupun praktisi berpengalaman — yang ingin bekerja bersama agen pemrograman secara produktif *dan* bertanggung jawab. Materinya disusun secara prosedural: dari instalasi, tur antarmuka, menghubungkan model, sesi coding pertama, mengendalikan agen dengan mode dan izin, memberi konteks proyek, hingga otomasi, kustomisasi plugin, dan praktik keselamatan. Setiap gambar diberi nomor, dan screenshot yang padat dilengkapi penanda bernomor beserta keterangannya.

Perlu diingat, DeepSeek Harness masih berstatus *developer preview* yang berkembang sangat cepat. Nama menu atau perilaku tertentu mungkin berubah setelah buku ini terbit. Karena itu, buku ini berusaha menekankan *cara berpikir* — merumuskan definisi selesai, merencanakan sebelum mengeksekusi, meninjau diff, dan membatasi izin — yang akan tetap relevan meskipun tombolnya berpindah tempat.

Semoga buku ini membantu Anda menjadikan AI sebagai rekan kerja yang dapat diandalkan: cepat ketika perlu cepat, hati-hati ketika perlu hati-hati. Kritik dan saran untuk penyempurnaan edisi berikutnya sangat penulis harapkan. Selamat ber-vibe coding — dan jangan lupa commit sebelum bereksperimen.

Oktober 2026

Syamsuddin

Daftar Isi

Bab 1 Mengenal DeepSeek Harness 1

1.1 Apa itu DeepSeek Harness? 1
1.2 Filosofi “semuanya adalah plugin” 1
1.3 Mengapa cocok untuk vibe coding? 2
1.4 Wajah-wajah DeepSeek Harness 2
1.5 Status developer preview dan keselamatan 3

Bab 2 Instalasi dan Menjalankan DSH 4

2.1 Prasyarat 4
2.2 Cara tercepat: lewat npm 4
2.3 Dari kode sumber 5
2.4 Mengenal perintah dsh 5
2.5 Di mana DSH menyimpan data? 6

Bab 3 Tur Antarmuka Web UI 8

3.1 Layar pertama 8
3.2 Menambahkan workspace 8
3.3 Anatomi layar utama 9
3.4 Menu akun dan menu More 10
3.5 Pengaturan umum 10

Bab 4 Menghubungkan Model AI 12

4.1 Memasang DeepSeek API key 12
4.2 Menambah penyedia pihak ketiga 13
4.3 Custom model API: gateway, relay, atau server sendiri 13
4.4 Memilih model dan tingkat penalaran 14
4.5 Konfigurasi lanjutan lewat YAML 14
4.6 Pemecahan masalah model 15

Bab 5 Sesi Pertama: Dari Ide ke Kode 16

5.1 Menulis perintah yang baik 16
5.2 Menyaksikan agen bekerja 16
5.3 To-do list: rencana yang bisa diawasi 17
5.4 Meninjau berkas yang ditulis 18
5.5 Memastikan test lulus 18
5.6 Iterasi: meminta perubahan 19
5.7 Membuka berkas di panel samping 20

5.8 Trajectory, statistik, dan ekspor 21

Bab 6 Mengendalikan Agen 23

6.1 Mode agen (preset) 23
6.2 Mode akses: seberapa jauh agen boleh menyentuh berkas 25
6.3 Panel persetujuan 25
6.4 Saat agen bertanya 26
6.5 Plan mode: rencanakan dulu, baru eksekusi 26
6.6 Perintah garis miring dan menu + 27
6.7 Menyebut berkas dengan @ 28

Bab 7 Memberi Konteks pada Agen 30

7.1 AGENTS.md dan CLAUDE.md 30
7.2 Skills: resep yang dapat dipanggil 30
7.3 Memori lintas sesi dengan server MCP 31

Bab 8 Otomasi: Headless, SDK, dan Tugas Terjadwal 33

8.1 Mode headless 33
8.2 SDK Python 34
8.3 Automation tasks: pengingat dan tugas berulang 34
8.4 Review otomatis dari webhook GitHub 36

Bab 9 Plugin dan Kustomisasi 37

9.1 Halaman Plugins 37
9.2 Memasang plugin lewat CLI 38
9.3 Indeks plugin (Discover) 38
9.4 Lapisan patch YAML 39
9.5 Creator mode: biarkan agen membuat plugin 39

Bab 10 Jaringan, Keamanan, dan Praktik Terbaik 40

10.1 Bekerja di balik proxy 40
10.2 Membuka Web UI di jaringan 40
10.3 Daftar periksa keselamatan 41
10.4 Sepuluh kebiasaan vibe coder yang efektif 41
10.5 Ke mana selanjutnya? 42

Lampiran A — Lembar Contekan 43

Lampiran B — Glosarium 44

Daftar Pustaka 46

Daftar Gambar

Gambar 2.1 Memeriksa versi lalu menyalakan Web UI. Alamat yang dicetak memuat token akses (disamarkan pada gambar). 5

Gambar 2.2 Keluaran dsh --help: sintaks umum, opsi peluncur, dan contoh pemakaian. 6

Gambar 3.1 Tampilan awal Web UI sebelum workspace dipilih. 8

Gambar 3.2 Dialog pemilihan workspace dengan jalur folder proyek yang diketik langsung. 9

Gambar 3.3 Bagian-bagian layar utama setelah workspace my-app terhubung (bilah samping di kiri, area composer di kanan). 9

Gambar 3.4 Menu akun: Settings, Feedback, dan Sign in. 10

Gambar 3.5 Menu More pada bilah samping: Browse other plugins dan Create your plugin. 10

Gambar 3.6 Halaman Settings → General. 11

Gambar 4.1 Kartu DeepSeek pada halaman Models. Sumber: dokumentasi resmi repositori (docs/user/guide). 12

Gambar 4.2 Menambah penyedia pihak ketiga. Petunjuk hijau menandakan variabel lingkungan kredensial terdeteksi. 13

Gambar 4.3 Formulir Custom model API. 14

Gambar 4.4 Pemilih model pada composer. 14

Gambar 5.1 Prompt yang memuat tujuan, batasan, dan definisi selesai. 16

Gambar 5.2 Jalannya satu giliran (turn) agen. 17

Gambar 5.3 To-do list awal: satu item sedang dikerjakan, tiga menunggu. 17

Gambar 5.4 Isi berkas src/store.js yang dibuat agen. 18

Gambar 5.5 Keluaran npm test: ketiga test lulus. 18

Gambar 5.6 Jawaban akhir: ringkasan berkas dan cara mencoba aplikasi. 19

Gambar 5.7 Giliran kedua: meminta perintah remove dan penomoran pada list. 19

Gambar 5.8 Diff perubahan pada src/cli.js: baris merah dihapus, hijau ditambahkan. 20

Gambar 5.9 Membuka src/cli.js di panel samping kanan. 21

Gambar 5.10 Tab Trajectory: linimasa peristiwa sesi di sebelah pratinjau berkas. 22

Gambar 5.11 Statistik sesi di bawah composer. 22

Gambar 5.12 Menu sesi: salin sebagai Markdown, unduh log, dan feedback. 22

Gambar 6.1 Pilihan mode agen. 23

Gambar 6.2 Halaman Agent presets. 24

Gambar 6.3 Tiga mode akses. 25

Gambar 6.4 Panel persetujuan. Perintah yang panjang ditampilkan dalam area bergulir agar tombol tetap terlihat. 26

Gambar 6.5 Panel pertanyaan dari agen. 26

Gambar 6.6 Kartu Plan review dengan tombol Request changes dan Approve. 27

Gambar 6.7 Rencana lengkap ditampilkan di panel samping melalui View full plan. 27

Gambar 6.8 Daftar perintah setelah mengetik /. 28

Gambar 6.9 Saran berkas setelah mengetik @src. 29
Gambar 8.1 Keluaran dsh headless --help. 33
Gambar 8.2 Halaman Automation tasks yang masih kosong. 35
Gambar 8.3 Dialog Create manually: nama, instruksi, tempat dijalankan, workspace, dan waktu. 35
Gambar 9.1 Halaman Plugins. 37
Gambar 9.2 Settings → Built-in plugins. 38
Gambar 10.1 Opsi dsh web --help: host, port, URL publik, host tepercaya, dan TLS. 41

BAB 1

Mengenal DeepSeek Harness

Sebelum mengetik perintah pertama, ada baiknya kita berkenalan dulu dengan ‘rekan kerja’ baru ini: apa ia sebenarnya, bagaimana ia dibangun, dan mengapa ia cocok untuk gaya kerja vibe coding.

1.1 Apa itu DeepSeek Harness?

DeepSeek Harness (disingkat **DSH**, dengan perintah `dsh`) adalah *agent harness* sumber terbuka yang dikembangkan oleh DeepSeek AI dan dirilis dengan lisensi MIT. Istilah *harness* secara harfiah berarti ‘tali kekang’: perangkat yang membuat tenaga besar bisa diarahkan. Dalam konteks ini, tenaga besarnya adalah model bahasa (LLM), sedangkan harness adalah kerangka yang memberi model itu tangan dan mata — kemampuan membaca berkas, menulis kode, menjalankan perintah terminal, mencari di web, menyusun rencana, dan mendelegasikan tugas — sambil tetap berada dalam batas izin yang Anda tentukan.

Hasilnya adalah **agen pemrograman** yang bekerja di dalam folder proyek Anda. Anda menjelaskan apa yang ingin dibangun dengan bahasa sehari-hari; agen menyusun langkah, menulis berkas, menjalankan test, lalu melaporkan hasilnya. Anda tetap memegang kemudi: menyetujui tindakan berisiko, meninjau perubahan, dan memutuskan kapan pekerjaan dianggap selesai.

1.2 Filosofi “semuanya adalah plugin”

Ciri paling khas DSH adalah arsitektur **everything-is-a-plugin**. DSH dibangun di atas **Cordis**, kerangka kerja plugin yang rancangannya dijelaskan dalam makalah *A Programming Paradigm for Spatiotemporal Composability*. Hampir setiap kemampuan — penyedia model, alat `bash`, pengelola berkas, Web UI, penjadwal, hingga mode perencanaan — adalah plugin yang dapat diaktifkan, dinonaktifkan, dikonfigurasi, atau diganti.

Bagi pengguna, konsekuensinya sederhana namun penting:

- **Profil** adalah tumpukan lapisan konfigurasi plugin. Profil `web` menghidupkan antarmuka peramban, profil `headless` menjalankan satu tugas lalu keluar, profil `sdk` melayani program Python.
- Konfigurasi ditulis dalam berkas YAML `cordis.patch.yml` — tidak perlu mengubah kode sumber untuk menyesuaikan perilaku.
- Fitur baru dapat dipasang sebagai paket plugin, bahkan dibuat oleh agen itu sendiri melalui **Creator mode**.

1.3 Mengapa cocok untuk vibe coding?

Vibe coding adalah gaya membangun perangkat lunak dengan mendeskripsikan maksud dalam bahasa alami dan membiarkan AI menulis sebagian besar kodenya, sementara programmer berperan sebagai pengarah, penguji, dan pengambil keputusan. Gaya ini menyenangkan sekaligus berisiko: cepat ketika semuanya lancar, membingungkan ketika agen tersesat. DSH menyediakan sejumlah ‘pagar pengaman’ yang membuat vibe coding lebih terkendali:

Kebutuhan vibe coder	Fitur DSH yang menjawabnya
Melihat apa yang sedang dikerjakan agen	Daftar langkah (*work details*), kartu alat, dan to-do list yang diperbarui langsung
Meninjau perubahan kode	Kartu *Write/Edit* dengan diff berwarna dan pratinjau berkas di panel samping
Mencegah tindakan berbahaya	Mode akses (Read Only, Workspace Write, Full access) dan panel persetujuan
Merencanakan sebelum menulis kode	Plan mode (`/plan`) dengan kartu tinjauan rencana
Memberi konteks proyek	`AGENTS.md` / `CLAUDE.md` , skills, dan sebutan berkas `@`
Bekerja tanpa layar	Mode `headless` , SDK Python, dan tugas otomatis terjadwal

1.4 Wajah-wajah DeepSeek Harness

Satu inti yang sama dapat dijalankan dalam beberapa ‘wajah’ atau mode masuk. Buku ini berfokus pada Web UI, tetapi wajah lain dibahas pada Bab 8.

Wajah	Perintah	Kegunaan
Web UI	`dsh web`	Antarmuka percakapan di peramban (default `http://127.0.0.1:3080`)
Headless	`dsh headless "tugas"`	Menjalankan satu tugas, mencetak jawaban, lalu keluar
SDK	`dsh --profile sdk`	Melayani klien SDK (mis. Python) lewat JSON-RPC stdio
ACP	`dsh --profile acp`	Melayani klien otomasi lewat Agent Client Protocol
Desktop	Aplikasi Electron	Paket aplikasi untuk Windows dan macOS (Apple Silicon maupun Intel)

1.5 Status developer preview dan keselamatan

⚠️ Baca sebelum memulai

DeepSeek Harness masih berstatus **developer preview** dan berkembang sangat cepat; pengembangnya menegaskan akan ada perubahan yang memutus kompatibilitas. Dokumen `SAFETY.md` di repositori menyatakan bahwa perangkat lunak ini belum menjalani audit keamanan, dapat mengeksekusi kode buatan model, memuat plugin pihak ketiga, serta mengakses jaringan, proses, kredensial, dan berkas yang tersedia baginya.

Rekomendasi resmi: jalankan dengan hak akses seminimal mungkin, utamakan mesin virtual atau kontainer sekali pakai, simpan cadangan berkas, jangan mengekspos kredensial sensitif, dan tinjau plugin serta perintah sebelum mengizinkannya berjalan.

📌 Tentang versi dalam buku ini

Seluruh screenshot diambil dari Web UI yang dibangun langsung dari kode sumber repositori proyek (versi `0.2.2-alpha.2`). Karena proyek bergerak cepat, nama menu atau tata letak di versi Anda mungkin sedikit berbeda. Prinsip kerjanya tetap sama.

BAB 2

Instalasi dan Menjalankan DSH

Ada dua jalan menuju DSH: jalan tol lewat npm dan jalan setapak lewat kode sumber. Keduanya berakhir di tempat yang sama — sebuah Web UI di peramban Anda.

2.1 Prasyarat

- **Node.js** terpasang. Untuk menjalankan dari kode sumber, repositori mensyaratkan Node.js **versi 24.9 atau lebih baru** dan **pnpm**.
- **Git**, karena agen bekerja paling baik di dalam repositori Git (akar proyek dikenali dari folder `.git`).
- **Kunci API** model, misalnya DeepSeek API key dari `platform.deepseek.com`, atau penyedia lain (Bab 4).
- Sistem operasi macOS (Apple Silicon atau Intel), Linux, atau Windows.

> 💡 **Untuk pengguna Mac Intel dan Apple Silicon**
>
> Jalur `npx` dan jalur kode sumber berjalan di mana pun Node.js berjalan, sehingga cocok untuk Mac Intel (x64) maupun Apple Silicon (arm64). Skrip pengemasan aplikasi Desktop di repositori juga menyediakan target `mac-x64` dan `mac-arm64`. Sebaliknya, wheel SDK Python saat ini hanya terdaftar untuk macOS 14+ pada arm64 — pengguna Mac Intel sebaiknya memakai Web UI atau mode headless.

2.2 Cara tercepat: lewat npm

Buka terminal di folder proyek Anda, lalu jalankan:

```bash
cd ~/my-app
npx @deepseek-ai/dsh web
```

Perintah ini menyalakan server Web UI di `http://127.0.0.1:3080` dan, untuk peluncuran lokal, langsung membukanya di peramban bawaan. Tambahkan `--no-open` bila Anda hanya ingin menyalakan server. Saat dijalankan lewat SSH, DSH hanya mencetak URL karena alamat yang diteruskan dimiliki klien SSH atau editor Anda.



Gambar 2.1 Memeriksa versi lalu menyalakan Web UI. Alamat yang dicetak memuat token akses (disamarkan pada gambar).

> ⚠️ **Jaga token Anda**
>
> URL yang dicetak berisi parameter `token=…` yang memberi akses ke sesi Anda. Jangan membagikannya di tangkapan layar atau obrolan publik.

2.3 Dari kode sumber

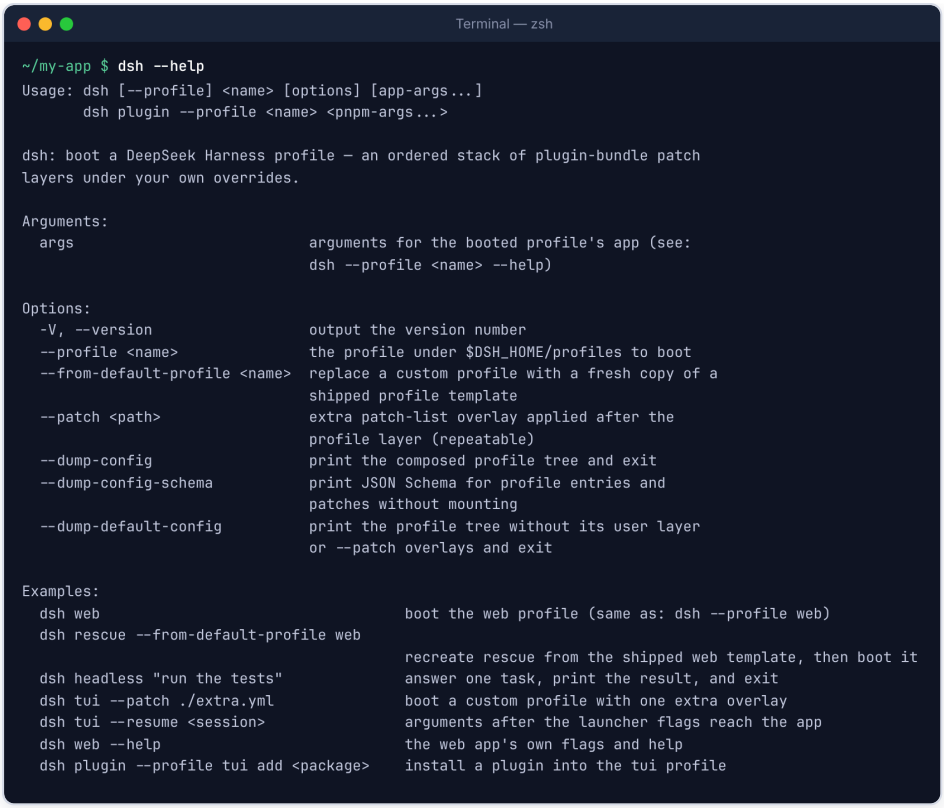
Jalur ini cocok bila Anda ingin mencoba fitur terbaru atau ikut berkontribusi.

```bash
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm dsh web
```

`pnpm run build` menyiapkan artefak repositori; `pnpm dsh web` memakai artefak yang sudah dibangun tanpa membangun ulang. Untuk pengembangan antarmuka, `pnpm run dev:web` membangun, menyajikan, dan membangun ulang bundel klien setiap kali sumber diedit.

2.4 Mengenal perintah dsh

Semua mode dijalankan lewat satu peluncur, `dsh`. Tanda (*flag*) milik peluncur ditulis lebih dulu; semua yang tidak dikenali diteruskan ke aplikasi profil yang dijalankan.



Gambar 2.2 Keluaran `dsh --help`: sintaks umum, opsi peluncur, dan contoh pemakaian.

Opsi	Fungsi
`--profile <nama>`	Memilih profil di bawah `$DSH_HOME/profiles`
`--patch <berkas>`	Menambahkan lapisan konfigurasi ekstra (dapat diulang)
`--dump-config`	Mencetak konfigurasi gabungan tanpa menjalankannya
`--from-default-profile <templat>`	Membuat ulang profil kustom dari templat bawaan

2.5 Di mana DSH menyimpan data?

DSH memakai folder rumah `$DSH_HOME` (default `~/.dsh`). Isinya yang penting:

- `profiles/<nama>/package.json` — manifes profil, termasuk daftar *bundle* plugin. Profil `web`, misalnya, menumpuk `@deepseek-ai/dsh-base` dan `@deepseek-ai/dsh-web-app`.
- `profiles/<nama>/cordis.patch.yml` — lapisan konfigurasi milik Anda untuk profil itu.
- `cordis.patch.yml` di akar `$DSH_HOME` — lapisan yang berlaku untuk semua profil.
- `.credentials.yaml` — tempat kunci API disimpan (pengaturan hanya menyimpan referensinya).

Urutan penyusunan konfigurasi: patch tiap bundle → `cordis.patch.yml` profil → `cordis.patch.yml` di rumah DSH → overlay `--patch`. Lapisan yang lebih belakang menimpa yang lebih depan.

> 💡 **Folder kerja default**
>
> Direktori tempat Anda menjalankan `dsh` menjadi lokasi berkas default. Namun Web UI yang baru belum memiliki workspace terpilih sampai Anda menambahkannya (Bab 3).

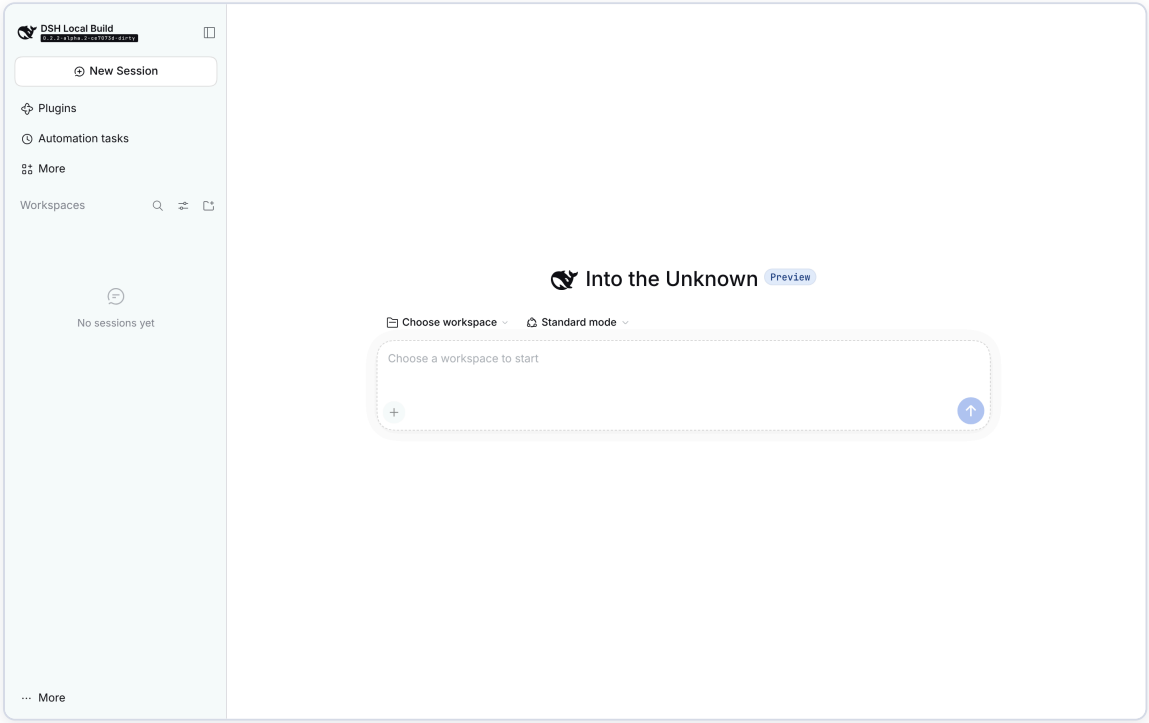
BAB 3

Tur Antarmuka Web UI

Web UI adalah ‘ruang kerja’ utama Anda bersama agen. Mari berkeliling sebentar supaya tidak ada tombol yang terasa asing.

3.1 Layar pertama

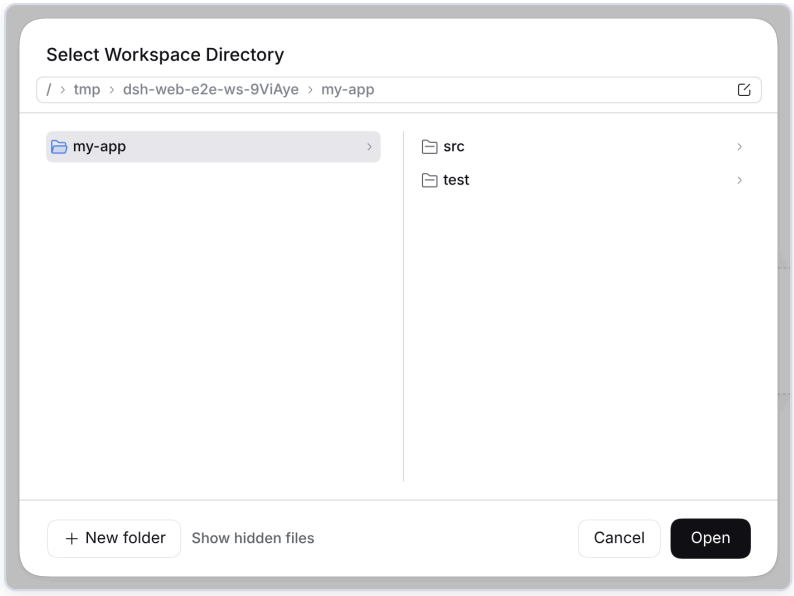
Saat pertama dibuka, Web UI menampilkan sambutan dan kotak perintah yang masih terkunci karena belum ada workspace. Langkah pertama Anda adalah memilih folder proyek.



Gambar 3.1 Tampilan awal Web UI sebelum workspace dipilih.

3.2 Menambahkan workspace

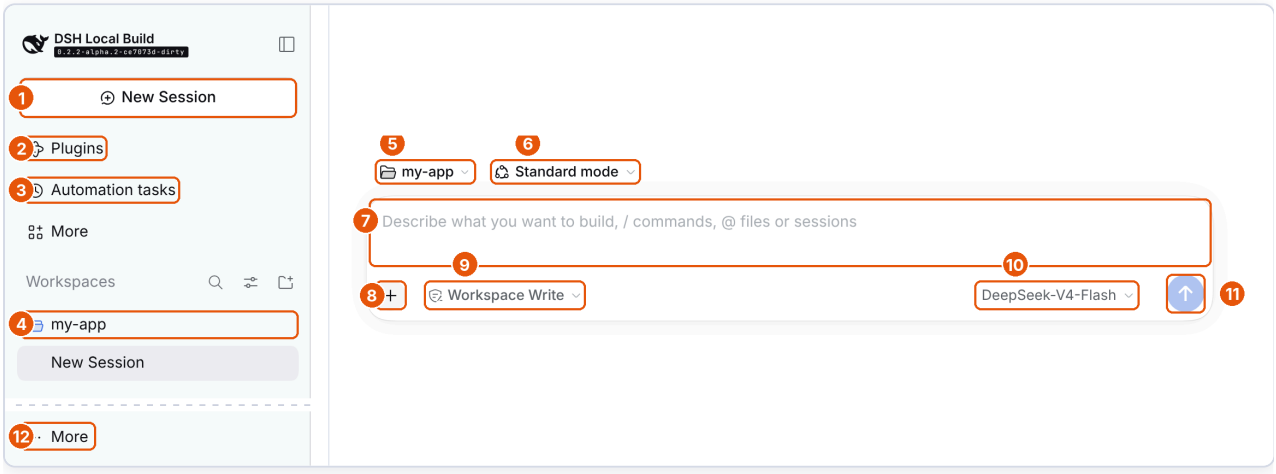
Klik ikon **Add workspace** (folder bertanda plus) di bagian *Workspaces* pada bilah samping. Dialog **Select Workspace Directory** akan muncul. Anda dapat menelusuri folder, membuat folder baru dengan **New folder**, menampilkan berkas tersembunyi, atau mengetik jalur secara langsung melalui ikon edit di kanan atas.



Gambar 3.2 Dialog pemilihan workspace dengan jalur folder proyek yang diketik langsung.

Setelah menekan **Open**, folder tersebut terdaftar sebagai workspace dan kotak perintah aktif.

3.3 Anatomi layar utama

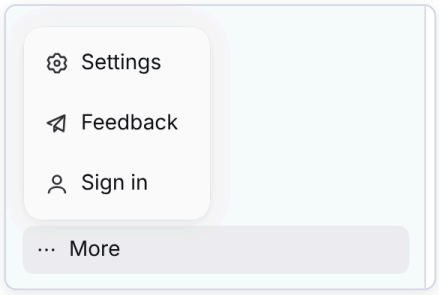


Gambar 3.3 Bagian-bagian layar utama setelah workspace `my-app` terhubung (bilah samping di kiri, area composer di kanan).

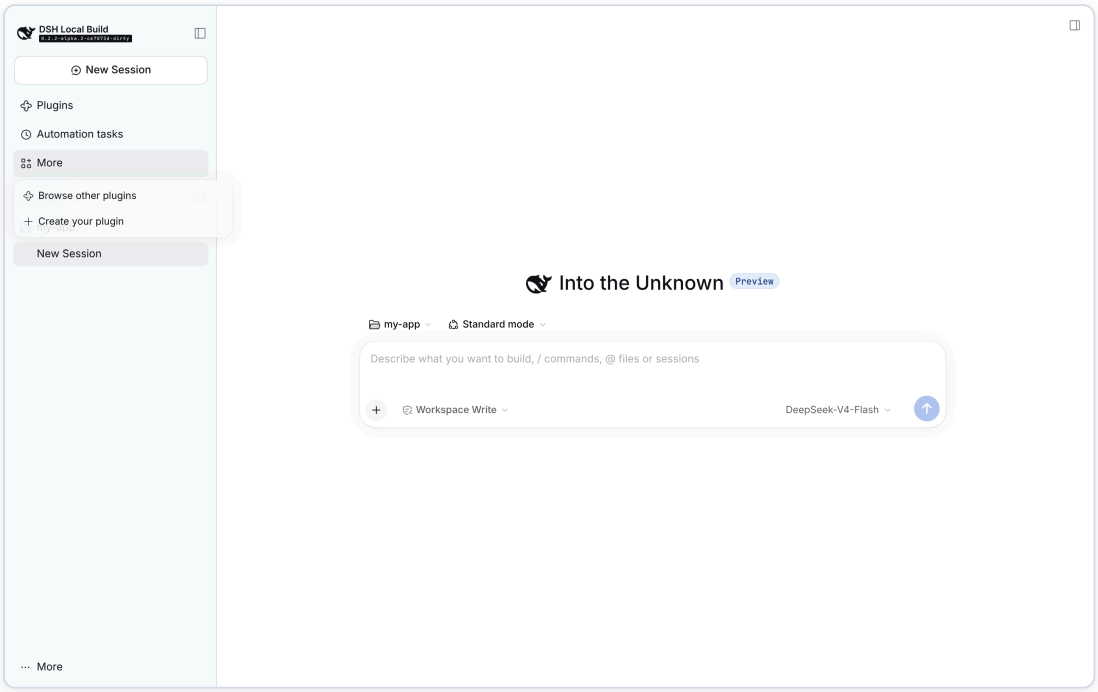
1. **New Session** — memulai percakapan (sesi) baru.
2. **Plugins** — mengelola plugin bawaan, eksperimental, dan terpasang (Bab 9).
3. **Automation tasks** — daftar tugas terjadwal (Bab 8).
4. **Workspaces** — daftar workspace beserta sesi-sesinya.
5. **Pemilih workspace** — workspace tempat sesi baru bekerja.
6. **Mode agen** (preset) — Standard, PTC, Minimal, atau Creator (Bab 6).
7. **Composer** — tempat mengetik perintah; mendukung `/` untuk perintah dan `@` untuk berkas atau sesi.
8. **Tombol +** — menambah berkas, goal, plan, feedback, dan perintah lain.
9. **Mode akses** — batas izin agen terhadap berkas (Bab 6).
10. **Pemilih model** — model yang dipakai sesi (Bab 4).
11. **Send message** — mengirim perintah (atau tekan Enter).
12. **Menu akun** — Settings, Feedback, dan Sign in.

3.4 Menu akun dan menu More

Menu akun di pojok kiri bawah membuka **Settings**, mengirim **Feedback**, atau masuk dengan akun DeepSeek (**Sign in**). Sementara itu, menu **More** di bagian atas bilah samping memuat pintasan untuk menjelajah plugin lain dan membuat plugin Anda sendiri.



Gambar 3.4 Menu akun: Settings, Feedback, dan Sign in.



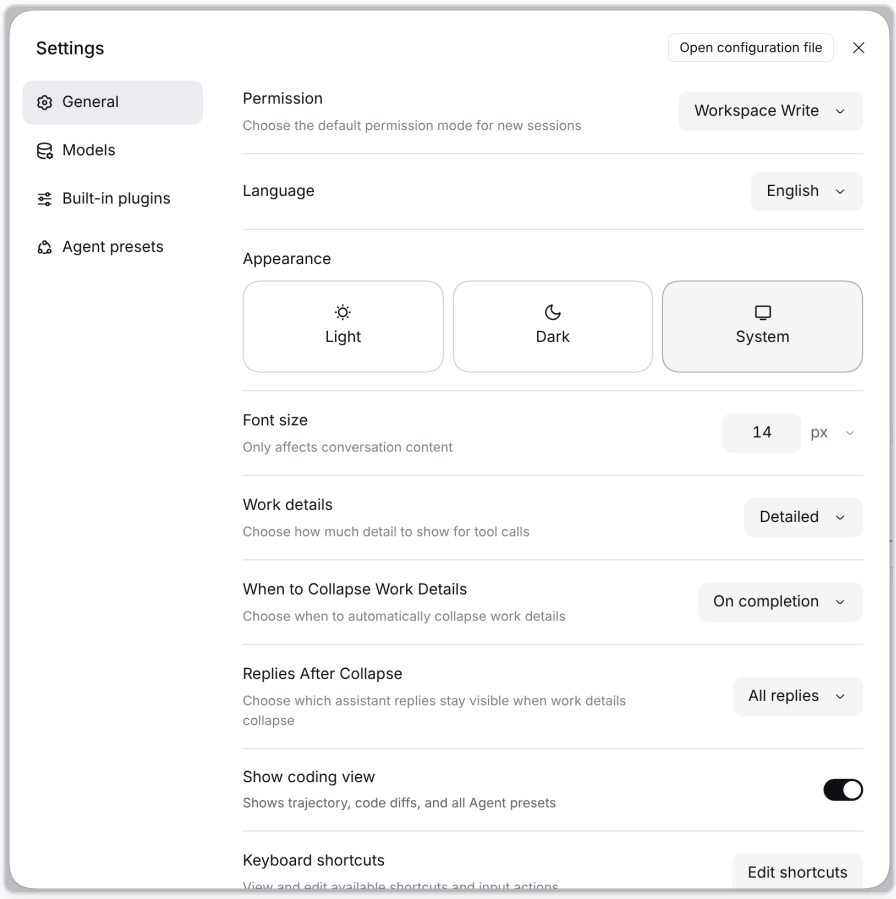
Gambar 3.5 Menu **More** pada bilah samping: *Browse other plugins* dan *Create your plugin*.

3.5 Pengaturan umum

Buka **Settings** → **General** untuk menyesuaikan pengalaman kerja. Beberapa yang paling relevan bagi vibe coder:

- **Permission** — mode akses default untuk sesi baru.
- **Appearance** dan **Font size** — tema terang/gelap/sistem dan ukuran huruf percakapan.
- **Work details** dan **When to collapse work details** — seberapa rinci langkah agen ditampilkan dan kapan diciutkan.

- **Show coding view** — menampilkan trajectory, diff kode, dan seluruh preset agen (termasuk PTC dan Minimal).
- **Keyboard shortcuts** — melihat dan mengubah pintasan papan ketik.



Gambar 3.6 Halaman Settings → General.

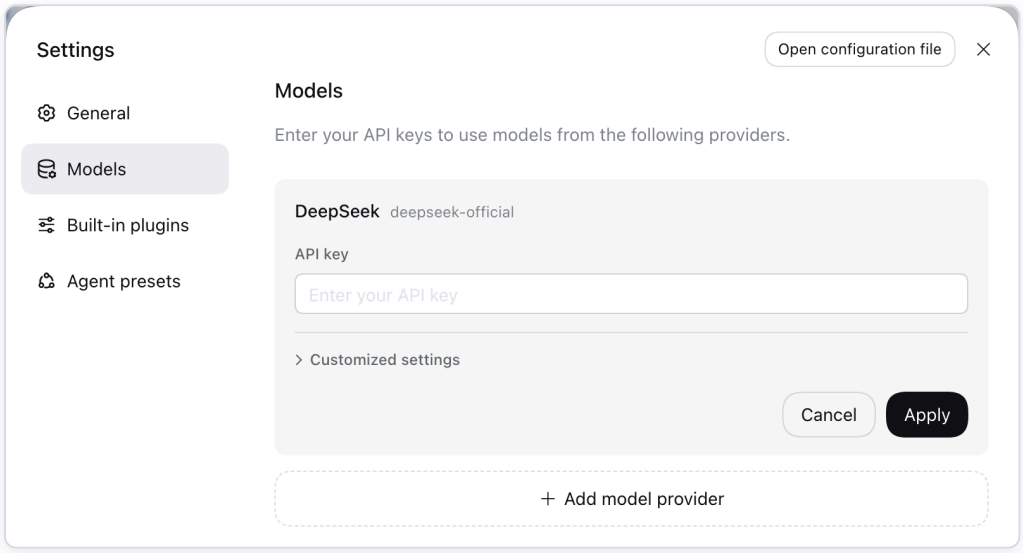
BAB 4

Menghubungkan Model AI

Agen tanpa model ibarat mobil tanpa mesin: tampak keren, tetapi tidak ke mana-mana. Bab ini memasang ‘mesin’ tersebut.

4.1 Memasang DeepSeek API key

1. Buka **Settings** → **Models**.
2. Pada kartu **DeepSeek** (`deepseek-official`), isi kolom **API key**.
3. Tekan **Apply**. Model langsung dapat dipakai tanpa memulai ulang server.



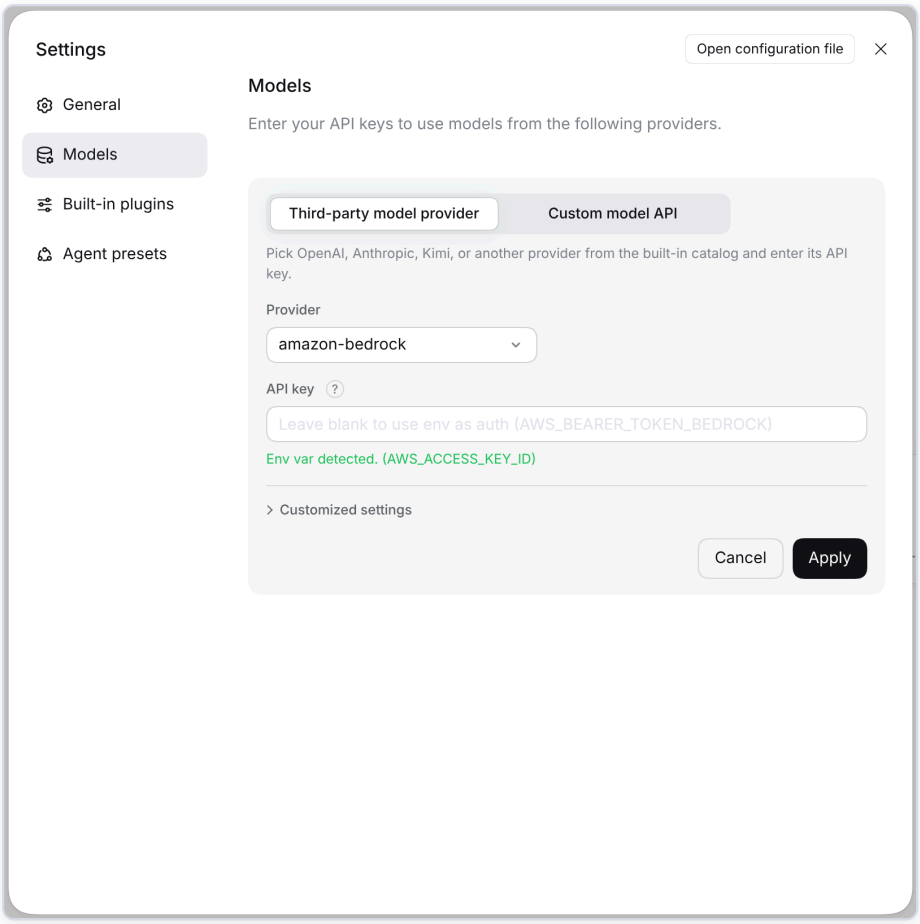
Gambar 4.1 Kartu DeepSeek pada halaman Models. *Sumber: dokumentasi resmi repositori (docs/user/guide).*

Kunci bersifat *write-only*: setelah disimpan, halaman hanya menerima deskriptor yang disamarkan, bukan kunci aslinya. Kunci disimpan di `$DSH_HOME/.credentials.yaml`. Dengan kunci tersimpan, daftar model DeepSeek mengikuti daftar yang dipublikasikan API DeepSeek dan diperbarui di latar belakang kira-kira setiap sepuluh menit selama dipakai.

> **💡 Alternatif lewat variabel lingkungan**
>
> Untuk mode headless atau CI, Anda dapat mengekspor `DEEPSEEK_API_KEY` (dan opsional `DEEPSEEK_BASE_URL`) sebelum menjalankan `dsh`, atau menaruhnya di `$DSH_HOME/.env`.

4.2 Menambah penyedia pihak ketiga

Pilih **Add model provider**. Kartu terbuka pada tab **Third-party model provider**: pilih penyedia yang sudah dikenal DSH — misalnya `anthropic`, `openai`, `moonshotai` (Kimi), atau `zai` (GLM) — isi kuncinya, lalu simpan. Katalog bawaan menyediakan endpoint, protokol, dan daftar model.



Gambar 4.2 Menambah penyedia pihak ketiga. Petunjuk hijau menandakan variabel lingkungan kredensial terdeteksi.

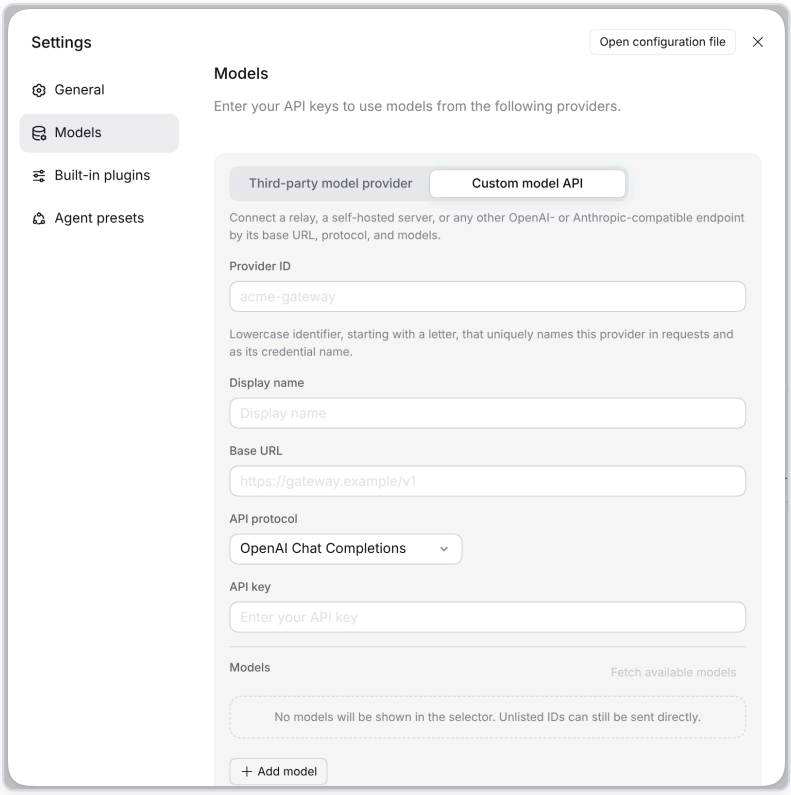
Jika kolom kunci dibiarkan kosong, DSH memakai kredensial tersimpan atau mencoba variabel lingkungan yang tercantum. Petunjuk hijau berarti variabel terdeteksi; petunjuk kuning berarti belum ada. Deteksi ini tidak memvalidasi kunci maupun menguji koneksi.

4.3 Custom model API: gateway, relay, atau server sendiri

Untuk gateway perusahaan, relay, atau server yang Anda hosting sendiri, pindah ke tab **Custom model API**. Isi:

- **Provider ID** — huruf kecil dan **permanen** (dipakai di log sesi dan referensi kredensial). Untuk mengganti nama, buat penyedia baru lalu hapus yang lama.
- **Display name** dan **Base URL**.
- **API protocol** — OpenAI Chat Completions, OpenAI Responses, atau Anthropic Messages. Satu penyedia hanya berbicara satu protokol.

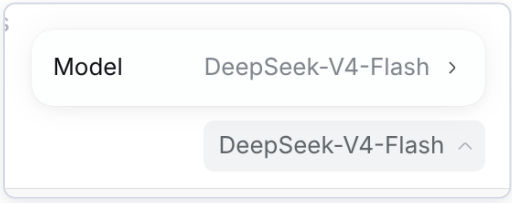
- **API key** dan minimal satu **model**. Tombol **Fetch available models** dapat menanyakan daftar model ke endpoint.



Gambar 4.3 Formulir Custom model API.

4.4 Memilih model dan tingkat penalaran

Penyedia yang sudah dikonfigurasi muncul di pemilih model pada composer. Memilih model juga menjadikannya default untuk sesi baru, sedangkan sesi yang sudah berjalan tetap memakai model yang tercatat di lognya. Untuk model yang mendukung penalaran, tersedia menu **Effort**; model DeepSeek menawarkan tingkat `off`, `low`, `high`, dan `max`.



Gambar 4.4 Pemilih model pada composer.

4.5 Konfigurasi lanjutan lewat YAML

Hal yang tidak tersedia di formulir — header, timeout, kebijakan retry, atau saklar kompatibilitas — diatur di `$DSH_HOME/profiles/web/cordis.patch.yml`. Tombol **Open configuration file** di header Settings

membuka berkas ini bila peramban berjalan di mesin yang sama. Adapter membaca ulang berkas pada permintaan berikutnya, jadi tidak perlu restart.

```yaml
- id: llm-pi-ai
  config:
    providers:
      my-gateway:
        apiKeyEnv: GATEWAY_API_KEY
        api: openai-completions
        baseURL: https://gateway.example/v1
        compat:
          supportsDeveloperRole: false
          maxTokensField: max_tokens
        models:
          - id: my-model
```

4.6 Pemecahan masalah model

Gejala	Solusi
`MISSING_CREDENTIAL`	Simpan kunci lewat halaman Models atau sediakan variabel lingkungan yang dirujuk.
`UNKNOWN_MODEL`	Pilih model yang terkonfigurasi atau tambahkan model yang hilang ke penyedia kustom.
Fetch models menghasilkan 401	Periksa kunci; bila endpoint tidak menyediakan `GET /models`, isi model secara manual.
Gateway menolak semua permintaan	Mulai dengan `compat.supportsDeveloperRole: false` dan `compat.maxTokensField: max_tokens`.
Menu Effort tidak muncul	Model yang ditambahkan manual belum mendeklarasikan tingkat penalaran; aktifkan **Reasoning levels** di *Model options*.

BAB 5

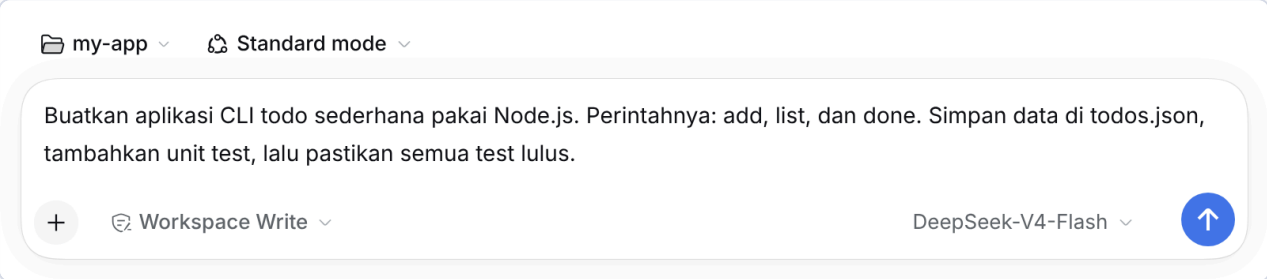
Sesi Pertama: Dari Ide ke Kode

Inilah inti vibe coding: menuliskan maksud, menyaksikan agen bekerja, lalu meninjau hasilnya. Kita akan membangun aplikasi CLI todo kecil dari nol.

> 📌 **Tentang contoh sesi**
>
> Sesi pada bab ini adalah contoh ilustratif yang dimuat ke Web UI asli DSH dalam format log sesinya, agar setiap tampilan dapat direproduksi tanpa kunci API. Antarmuka, kartu alat, diff, dan statistik yang tampak adalah rendering sebenarnya dari DSH; isi percakapannya disusun untuk keperluan tutorial.

5.1 Menulis perintah yang baik

Prompt yang efektif untuk agen pemrograman biasanya memuat tiga hal: **apa** yang dibangun, **batasan** (teknologi, berkas, gaya), dan **definisi selesai** (mis. ‘semua test lulus’).

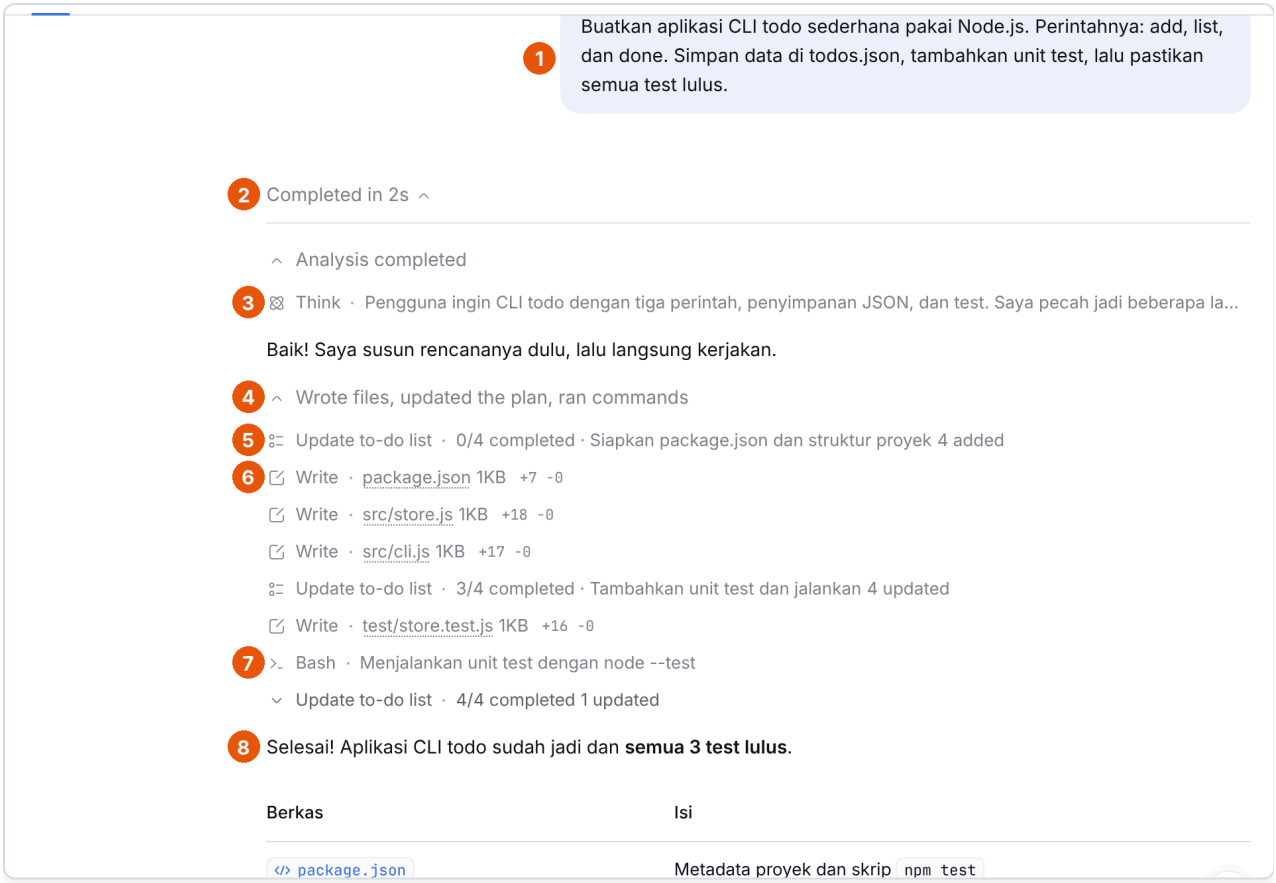


Gambar 5.1 Prompt yang memuat tujuan, batasan, dan definisi selesai.

> 💡 **Pola prompt vibe coding**
>
> **Tujuan** → “Buatkan aplikasi CLI todo…”
> **Batasan** → “pakai Node.js, simpan di todos.json”
> **Bukti selesai** → “tambahkan unit test, pastikan semua lulus”.
> Kalimat terakhir ini membuat agen memverifikasi pekerjaannya sendiri.

5.2 Menyaksikan agen bekerja

Setelah prompt dikirim, agen bekerja dalam beberapa *langkah*. Setiap langkah tercatat dan dapat dibuka untuk dilihat rinciannya.

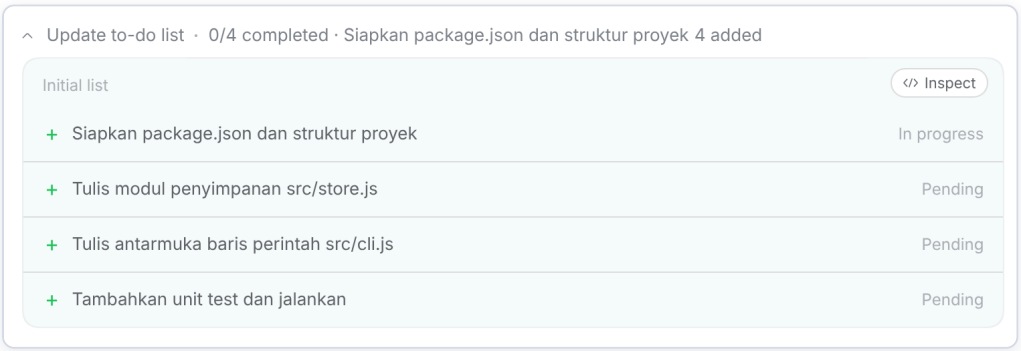


Gambar 5.2 Jalannya satu giliran (turn) agen.

1. Prompt Anda.
2. Ringkasan giliran: selesai beserta durasinya; klik untuk melipat atau membuka rincian.
3. **Think** — ringkasan penalaran model.
4. Kelompok tindakan: menulis berkas, memperbarui rencana, menjalankan perintah.
5. **Update to-do list** — rencana kerja yang dicatat agen.
6. **Write** — berkas yang dibuat, dengan jumlah baris yang ditambah/dihapus.
7. **Bash** — perintah terminal yang dijalankan (di sini: `npm test`).
8. Jawaban akhir agen.

5.3 To-do list: rencana yang bisa diawasi

Untuk tugas bertahap, agen mencatat rencananya memakai alat `todo_write`. Daftar ini diperbarui seiring kemajuan sehingga Anda bisa melihat bagian mana yang sedang dikerjakan.



Gambar 5.3 To-do list awal: satu item sedang dikerjakan, tiga menunggu.

5.4 Meninjau berkas yang ditulis

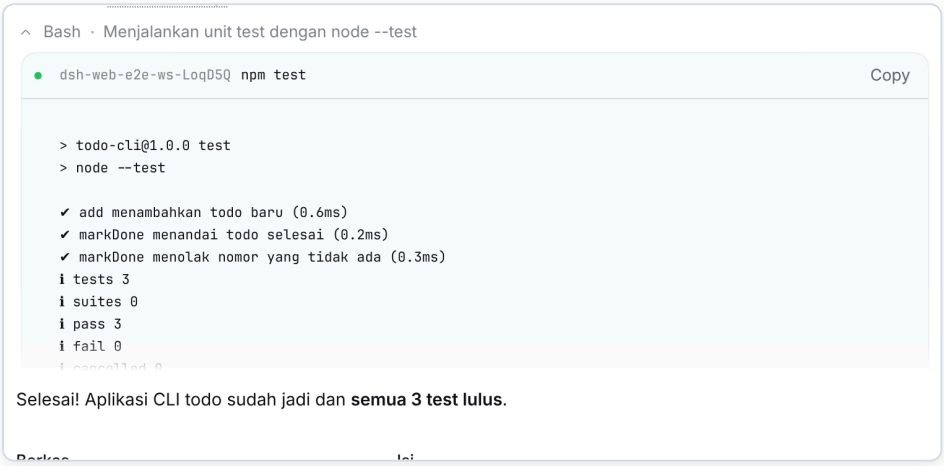
Klik kartu **Write** untuk melihat isi berkas yang dibuat dalam bentuk diff hijau (baris tambahan).



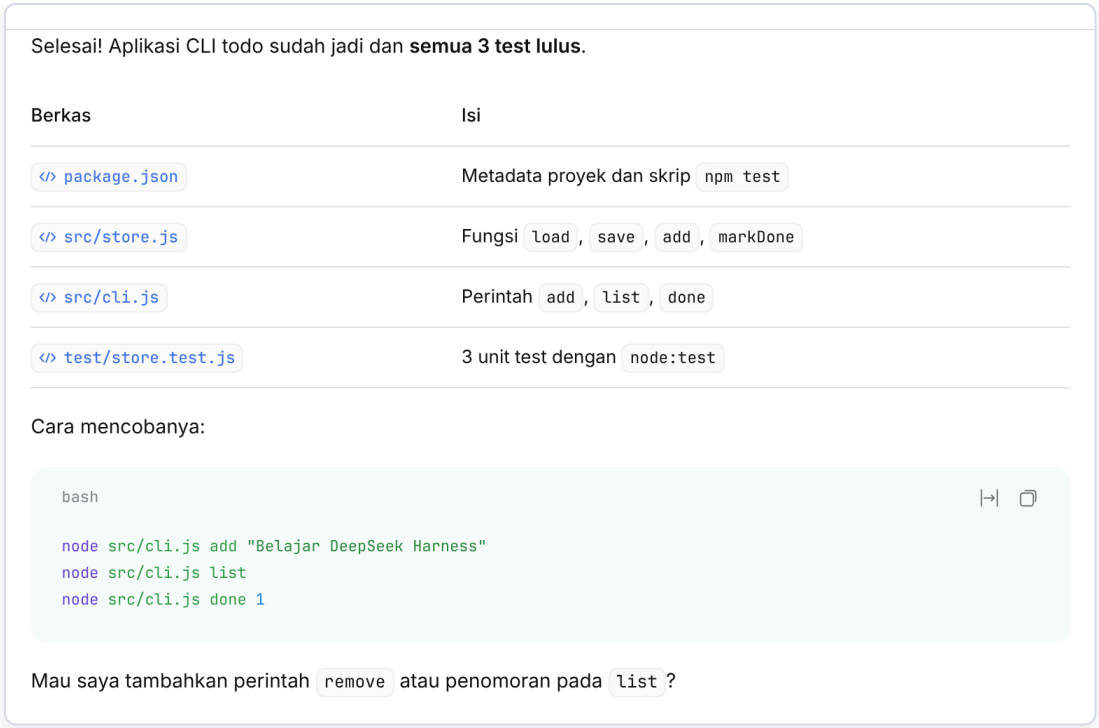
Gambar 5.4 Isi berkas `src/store.js` yang dibuat agen.

5.5 Memastikan test lulus

Kartu **Bash** menampilkan perintah beserta keluarannya. Inilah bukti bahwa ‘definisi selesai’ benar-benar terpenuhi, bukan sekadar klaim agen.



Gambar 5.5 Keluaran `npm test`: ketiga test lulus.



Selesai! Aplikasi CLI todo sudah jadi dan **semua 3 test lulus**.

Berkas	Isi
`package.json`	Metadata proyek dan skrip `npm test`
`src/store.js`	Fungsi `load` , `save` , `add` , `markDone`
`src/cli.js`	Perintah `add` , `list` , `done`
`test/store.test.js`	3 unit test dengan `node:test`

Cara mencobanya:

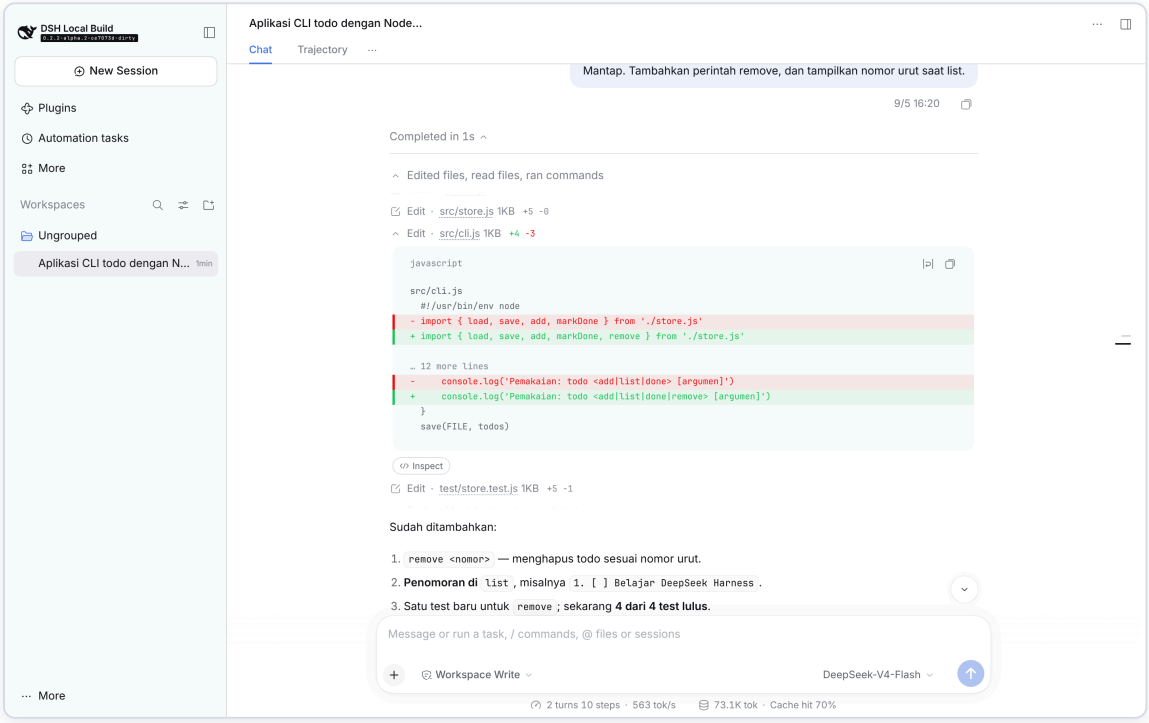
```bash
node src/cli.js add "Belajar DeepSeek Harness"
node src/cli.js list
node src/cli.js done 1
```

Mau saya tambahkan perintah `remove` atau penomoran pada `list` ?

Gambar 5.6 Jawaban akhir: ringkasan berkas dan cara mencoba aplikasi.

5.6 Iterasi: meminta perubahan

Vibe coding jarang selesai dalam satu kali jalan. Lanjutkan percakapan di sesi yang sama; agen sudah memiliki konteks berkas yang ia tulis.



Gambar 5.7 Giliran kedua: meminta perintah `remove` dan penomoran pada `list` .

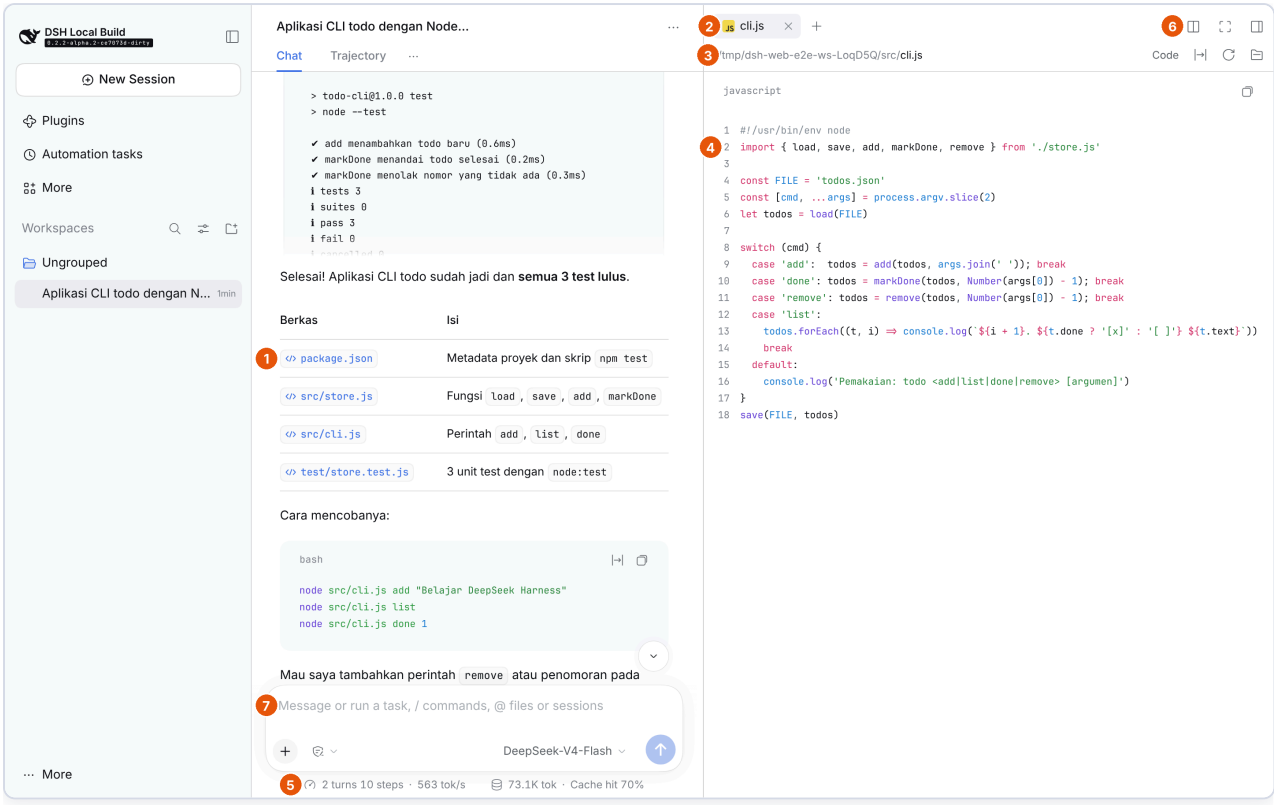
Kali ini agen membaca berkas lalu mengeditnya. Kartu **Edit** menampilkan diff merah-hijau yang memudahkan peninjauan.



Gambar 5.8 Diff perubahan pada `src/cli.js`: baris merah dihapus, hijau ditambahkan.

5.7 Membuka berkas di panel samping

Nama berkas pada jawaban agen dapat diklik untuk dibuka di panel kanan, lengkap dengan penomoran baris dan pewarnaan sintaks.

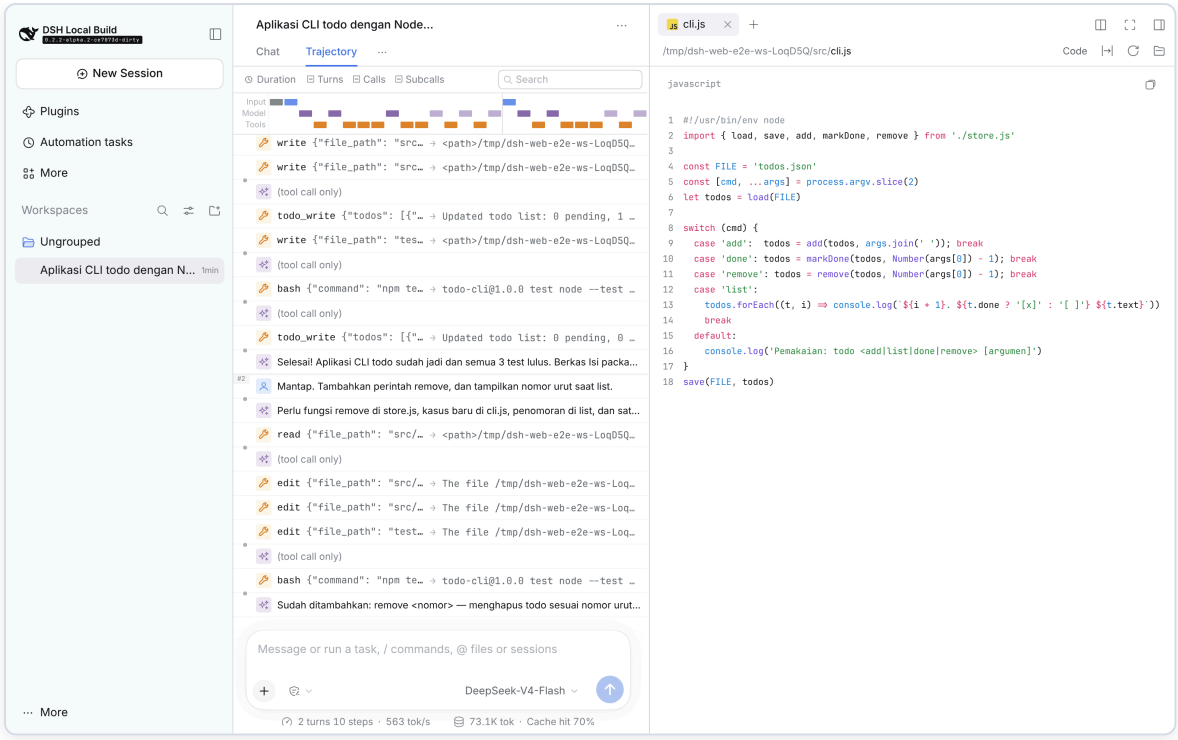


Gambar 5.9 Membuka `src/cli.js` di panel samping kanan.

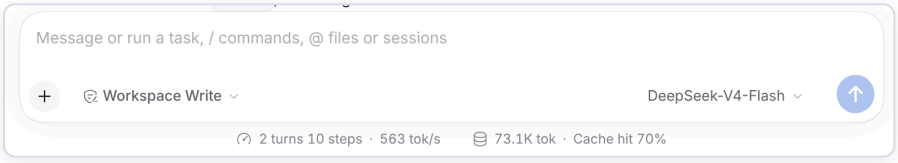
1. Tautan berkas di jawaban agen — klik untuk membuka.
2. Tab berkas yang terbuka; tombol + menambah tab.
3. Jalur lengkap berkas di workspace.
4. Isi berkas dengan nomor baris.
5. Statistik sesi: jumlah giliran dan langkah, kecepatan, token, dan cache hit.
6. Kontrol panel: split, layar penuh, tutup panel.
7. Composer untuk melanjutkan percakapan.

5.8 Trajectory, statistik, dan ekspor

Tab **Trajectory** menampilkan linimasa seluruh peristiwa sesi — panggilan alat, hasil, dan pesan — berguna untuk menelusuri ‘apa yang sebenarnya terjadi’. Di bawah composer, baris statistik merangkum giliran, langkah, token, dan rasio cache.

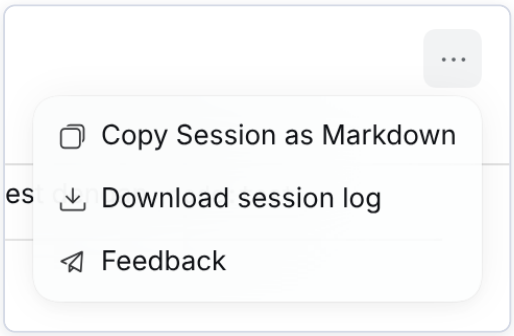


Gambar 5.10 Tab Trajectory: linimasa peristiwa sesi di sebelah pratinjau berkas.



Gambar 5.11 Statistik sesi di bawah composer.

Menu tiga titik di header sesi menyediakan **Copy Session as Markdown**, **Download session log**, dan **Feedback**. Di bawah setiap jawaban terdapat tombol salin, penilaian (suka/tidak suka), dan **Branch into a new conversation** untuk mencabangkan percakapan.



Gambar 5.12 Menu sesi: salin sebagai Markdown, unduh log, dan feedback.

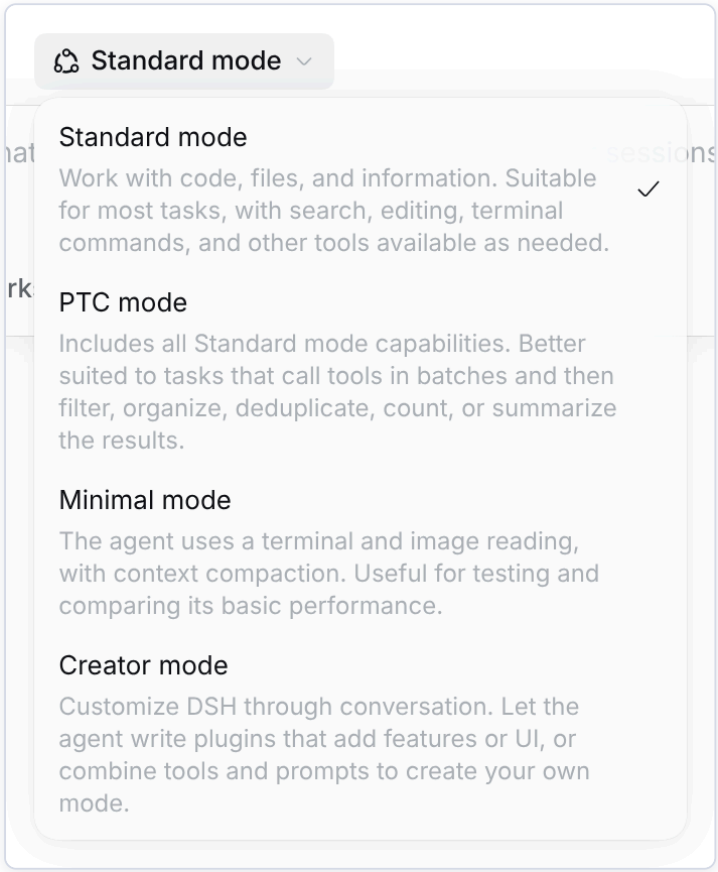
BAB 6

Mengendalikan Agen: Mode, Izin, dan Persetujuan

Agen yang cerdas tetap perlu rambu-rambu. Bab ini membahas tuas-tuas kendali yang membuat vibe coding tetap aman dan terarah.

6.1 Mode agen (preset)

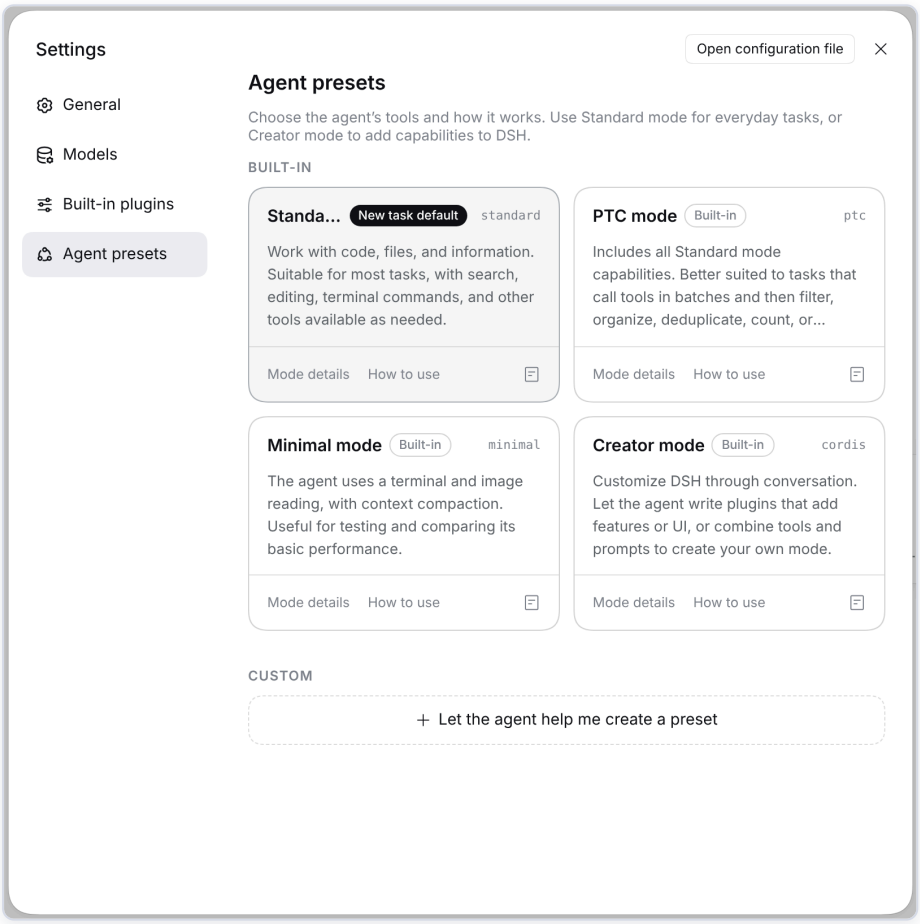
Pemilih mode di atas composer menentukan ‘kepribadian’ dan perangkat alat agen untuk sesi baru.



Gambar 6.1 Pilihan mode agen.

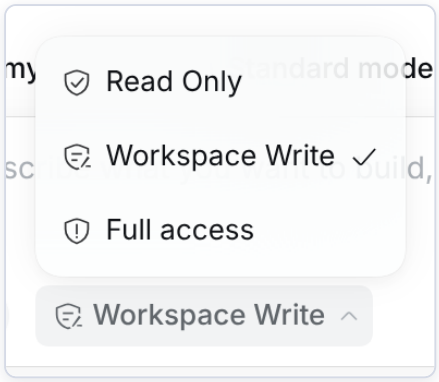
Mode	Kapan dipakai
Standard	Pekerjaan sehari-hari: kode, berkas, informasi; dilengkapi pencarian, penyuntingan, terminal, dan alat lain.
PTC	Semua kemampuan Standard, lebih cocok untuk tugas yang memanggil banyak alat sekaligus lalu menyaring, mengurutkan, menghitung, atau merangkum hasilnya (alat `run_code`).
Minimal	Hanya terminal dan pembacaan gambar, dengan pemadatan konteks; berguna untuk menguji dan membandingkan performa dasar.
Creator	Menyesuaikan DSH lewat percakapan: agen menulis plugin yang menambah fitur atau UI, atau meracik mode Anda sendiri.

PTC dan Minimal muncul saat **Show coding view** diaktifkan di Settings. Halaman **Settings → Agent presets** menampilkan seluruh preset, menandai default untuk tugas baru, dan menyediakan tombol *View configuration* untuk melihat daftar plugin yang dideklarasikan preset.



Gambar 6.2 Halaman Agent presets.

6.2 Mode akses: seberapa jauh agen boleh menyentuh berkas



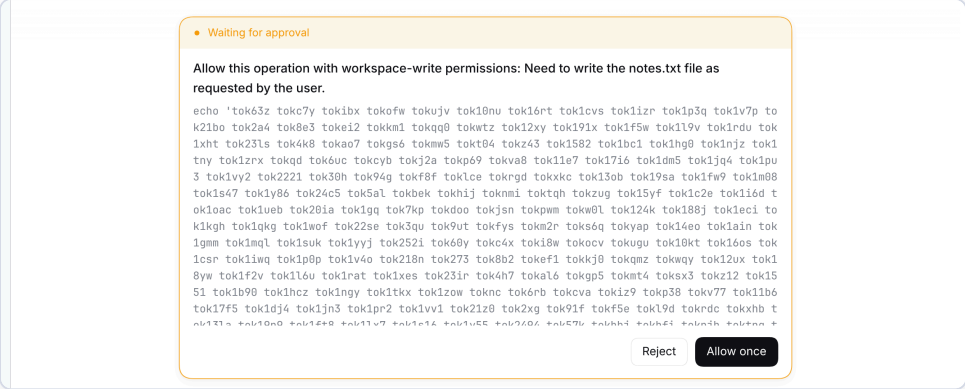
Gambar 6.3 Tiga mode akses.

Mode akses	Arti
Read Only	Agen hanya boleh membaca. Ini juga default aman (*fail-safe*) dari kebijakan sandbox.
Workspace Write	Agen boleh mengubah berkas di dalam workspace (dan direktori tambahan yang diizinkan).
Full access	Setara `danger-full-access` : tanpa pembatasan berkas oleh sandbox DSH. Gunakan hanya di lingkungan sekali pakai.

> ⚠️ **Sandbox bukan jaminan mutlak**
>
> Sandbox, persetujuan, dan kontrol izin mengurangi risiko tetapi tidak menjamin isolasi. Pembatasan yang benar sekalipun tidak melindungi sumber daya yang memang Anda izinkan untuk diakses.

6.3 Panel persetujuan

Ketika agen membutuhkan tindakan di luar izin aktif — misalnya menulis berkas saat mode Read Only — composer digantikan panel **Waiting for approval**. Panel menjelaskan permintaan dan menampilkan perintah yang akan dijalankan. Anda memilih **Reject** atau **Allow once**.

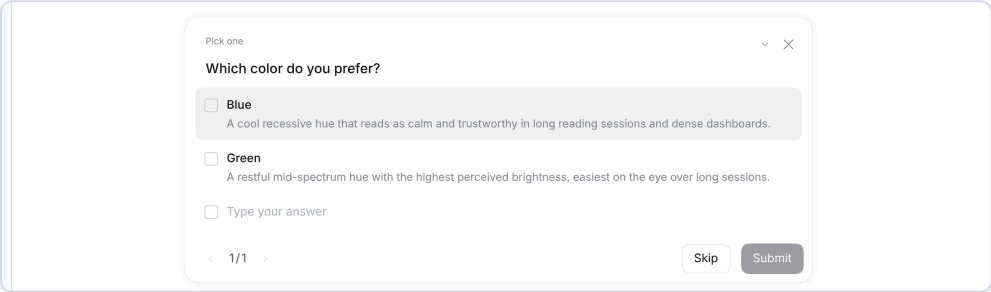


Gambar 6.4 Panel persetujuan. Perintah yang panjang ditampilkan dalam area bergulir agar tombol tetap terlihat.

> **💡 Kebiasaan baik**
>
> Baca perintahnya, bukan hanya penjelasannya. ‘Allow once’ hanya berlaku untuk satu tindakan itu dan tidak mengubah mode akses sesi.

6.4 Saat agen bertanya

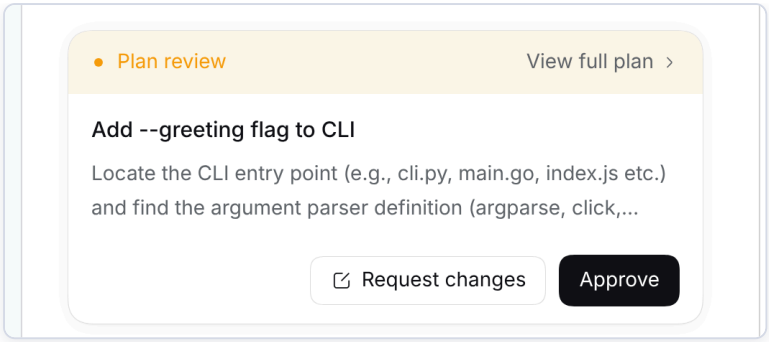
Bila instruksi ambigu, agen dapat bertanya memakai alat `ask_user_question`. Panel pertanyaan menampilkan pilihan (bisa lebih dari satu), kolom jawaban bebas, serta tombol **Skip** dan **Submit**.



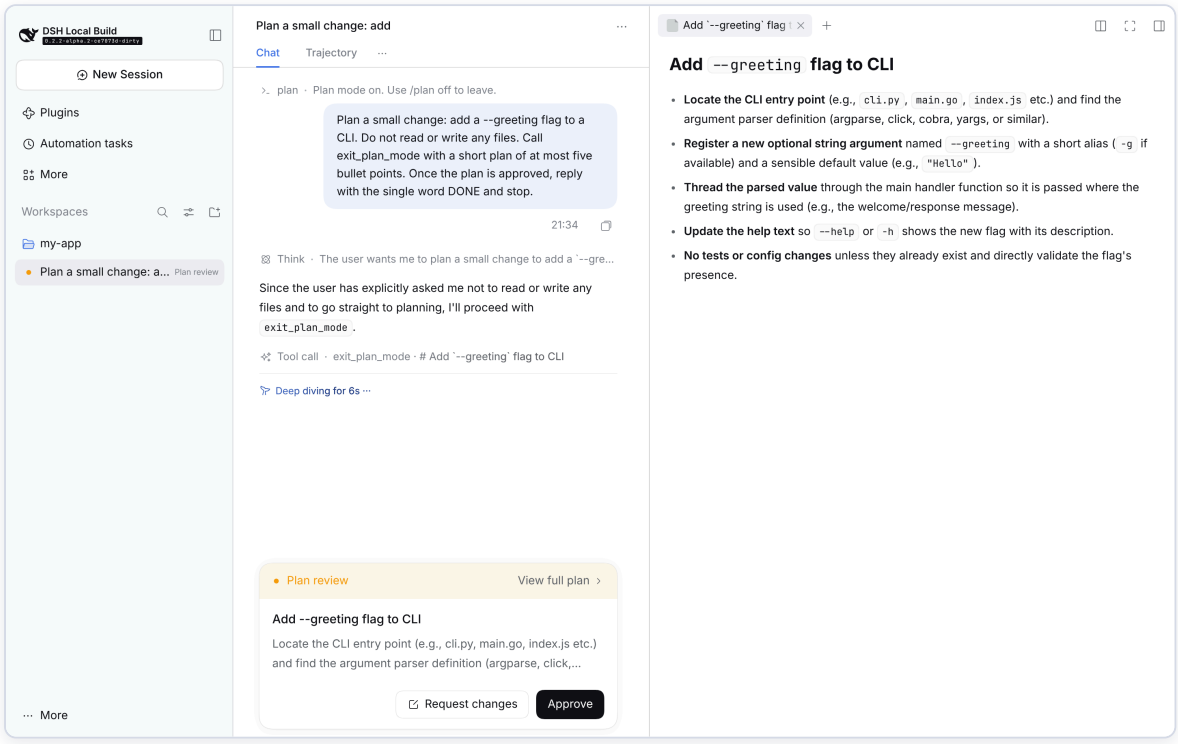
Gambar 6.5 Panel pertanyaan dari agen.

6.5 Plan mode: rencanakan dulu, baru eksekusi

Untuk perubahan yang lebih besar, minta agen menyusun rencana sebelum menyentuh kode. Ketik `/plan` diikuti tugas Anda. Agen akan menjelajah dan merancang, lalu menyerahkan rencana lewat kartu **Plan review**. Pilih **Approve** untuk melanjutkan eksekusi, atau **Request changes** untuk memberi masukan. Keluar dari plan mode dengan `/plan off`.



Gambar 6.6 Kartu Plan review dengan tombol Request changes dan Approve.

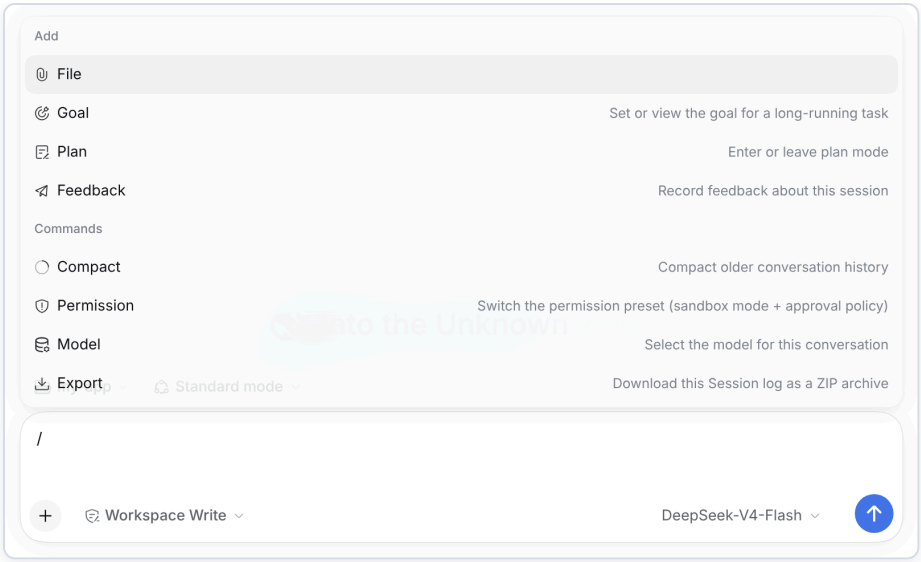


Gambar 6.7 Rencana lengkap ditampilkan di panel samping melalui *View full plan*.

> **📌 Plan mode adalah panduan, bukan kunci**
>
> Selama plan mode, semua alat tetap tersedia; perilaku perencanaan diarahkan oleh instruksi. Untuk pembatasan yang benar-benar ditegakkan, gunakan mode akses dan persetujuan.

6.6 Perintah garis miring dan menu +

Ketik `/` di composer, atau klik tombol +, untuk membuka daftar tindakan cepat.

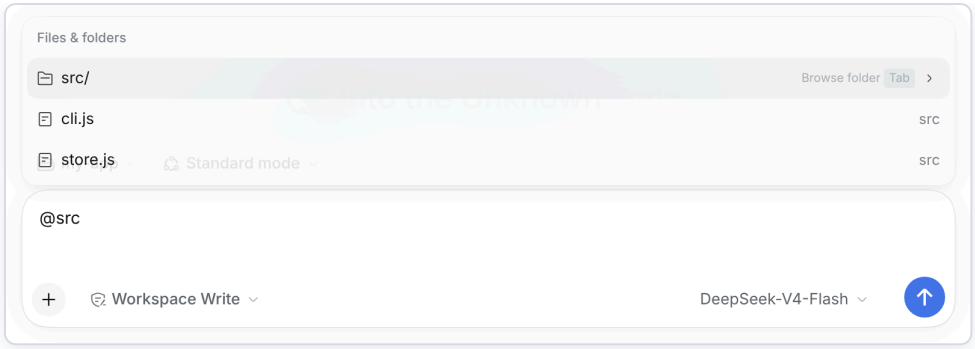


Gambar 6.8 Daftar perintah setelah mengetik `/`.

Item	Fungsi
File	Melampirkan berkas
Goal	Menetapkan atau melihat tujuan untuk tugas panjang
Plan	Masuk atau keluar plan mode
Feedback	Mencatat masukan tentang sesi
Compact	Memadatkan riwayat percakapan lama agar konteks lega
Permission	Mengganti preset izin (mode sandbox + kebijakan persetujuan)
Model	Memilih model untuk percakapan ini
Export	Mengunduh log sesi sebagai arsip ZIP

6.7 Menyebut berkas dengan @

Ketik `@` diikuti potongan nama untuk menyisipkan berkas, folder, atau bahkan sesi lain sebagai konteks. Ini cara paling cepat mengarahkan perhatian agen ke bagian kode yang tepat.



Gambar 6.9 Saran berkas setelah mengetik `@src`.

BAB 7

Memberi Konteks pada Agen

Agen yang paham konvensi proyek Anda akan jauh lebih jarang 'berimprovisasi'. Bab ini membahas tiga cara menanamkan pengetahuan: berkas instruksi, skills, dan memori.

7.1 AGENTS.md dan CLAUDE.md

Plugin `dsh-agent-instructions` (aktif secara default di `dsh-base`) memuat berkas instruksi yang kompatibel dengan `AGENTS.md` pada permintaan pertama sesi:

1. **Rantai global pengguna**: `$DSH_HOME/AGENTS.md`, lalu `~/.agents/AGENTS.md`.
2. **Rantai proyek**: setiap berkas kandidat dari akar proyek (folder yang memuat `.git`) turun hingga direktori kerja sesi, dari yang umum ke yang spesifik.

Kandidat dasarnya `AGENTS.md` dan `CLAUDE.md`; `AGENTS.local.md` dan `CLAUDE.local.md` menjadi lapisan tambahan lokal. Isi yang identik hanya ditampilkan sekali. Ketika agen membaca atau menulis berkas di folder yang lebih dalam, berkas instruksi di folder itu ikut dimuat pada permintaan berikutnya.

```markdown
# AGENTS.md — my-app

## Perintah
- Install: npm install
- Test: npm test (wajib lulus sebelum menyatakan selesai)

## Konvensi
- ES Modules, Node 24
- Pesan error dalam bahasa Indonesia
- Jangan menambah dependensi tanpa bertanya
```

> 💡 **Tulis singkat dan operasional**
>
> Isi AGENTS.md dengan hal yang *tidak bisa ditebak* dari kode: perintah build/test, konvensi tim, dan larangan. Anggaran byte membatasi setiap rantai, jadi berkas ringkas lebih aman daripada esai panjang.

7.2 Skills: resep yang dapat dipanggil

Skill adalah instruksi berpaket untuk jenis tugas tertentu. Skill ditulis sebagai folder `<nama>/SKILL.md` atau berkas datar `<nama>.md` dengan frontmatter YAML berisi `name` dan `description` (wajib). DSH memindai lokasi berikut dan memantau perubahannya tanpa perlu restart:

Sumber	Lokasi
Proyek (DSH)	`<proyek>/.dsh/skills`
Proyek (agents)	`<proyek>/.agents/skills`
Kustom	Direktori di konfigurasi `customSkillDirs`
Pengguna (DSH)	`$DSH_HOME/skills`
Pengguna (agents)	`~/.agents/skills`

```markdown
---
name: rilis-npm
description: Langkah merilis paket ke npm dengan changelog dan tag Git.
---
1. Jalankan npm test; hentikan bila gagal.
2. Perbarui CHANGELOG.md dari commit sejak tag terakhir.
3. npm version <patch|minor|major>, lalu git push --follow-tags.
```

Frontmatter opsional `disable-model-invocation: true` menyembunyikan skill dari model, sedangkan `user-invocable: false` menyembunyikannya dari perintah manusia.

7.3 Memori lintas sesi dengan server MCP

DSH dapat terhubung ke server **MCP** (Model Context Protocol) melalui plugin `@deepseek-ai/dsh-mcp-client`; alat-alatnya muncul dengan nama `mcp__<server>__<alat>`. Repositori menyertakan tiga contoh konfigurasi memori yang nonaktif secara default — Memorix, MCP Reference Memory, dan Engram — sebagai contoh interoperabilitas (bukan rekomendasi resmi).

```bash
npm install --global @modelcontextprotocol/server-memory@2026.7.4
dsh web --patch "$PWD/apps/cli/config/examples/mcp-memory/mcp-reference-memory.cordis.yml"
```

Untuk server MCP lain, salin baris generik berikut ke lapisan patch Anda dengan `id` dan `serverName` yang unik. Untuk server jarak jauh, gunakan `transport: streamable-http` beserta `url` dan `headers`.

```yaml
- insert:
    - id: memory-my-server
      name: '@deepseek-ai/dsh-mcp-client'
      config:
        serverName: my-memory
        transport: stdio
        command: my-memory-mcp
        args: []
        env: {}
        cwd: !!js process.cwd()
```

📌 Keamanan variabel lingkungan

Jembatan stdio sengaja membuang variabel lingkungan yang namanya biasanya menandakan kredensial serta semua variabel `DSH_*` sebelum menjalankan server anak. Tambahkan rahasia yang memang dibutuhkan lewat `config.env`, bukan ditulis langsung di YAML.

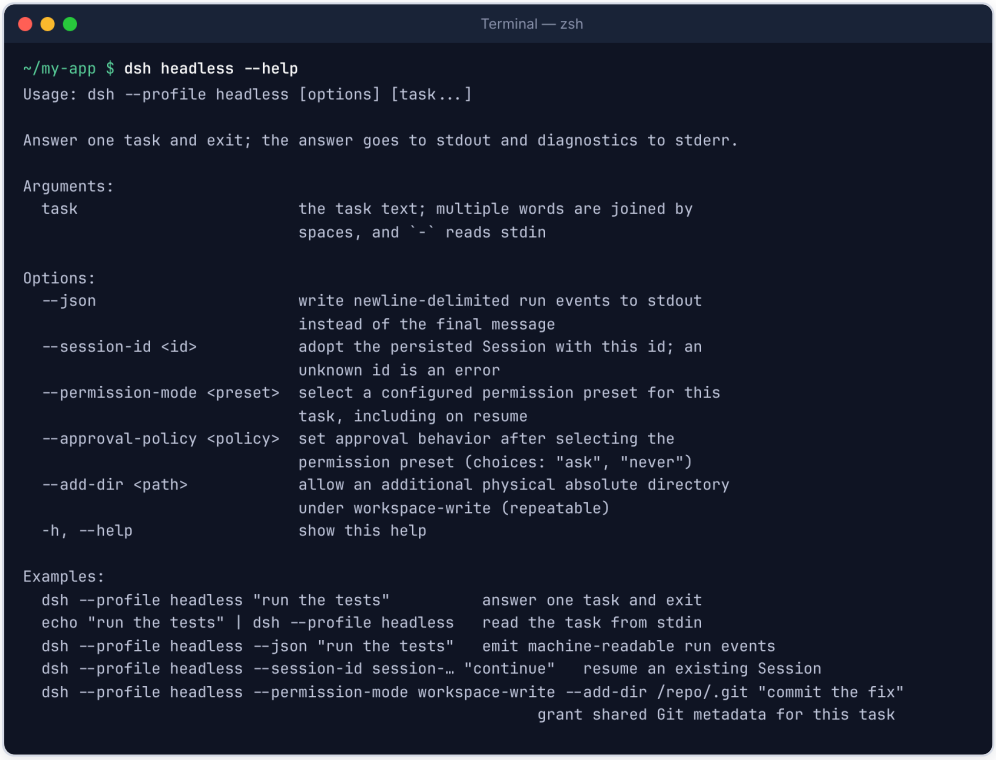
BAB 8

Otomasi: Headless, SDK, dan Tugas Terjadwal

Tidak semua pekerjaan butuh jendela peramban. Bab ini membawa agen ke skrip, pipeline CI, program Python, dan jadwal harian.

8.1 Mode headless

Mode headless menjalankan satu sesi baru yang tersimpan, mencetak jawaban akhir ke *stdout* (diagnostik ke *stderr*), lalu keluar.



Gambar 8.1 Keluaran `dsh headless --help`.

```bash
# satu tugas
dsh --profile headless "jalankan test dan rangkum yang gagal"

# tugas dari stdin
echo "perbaiki lint error di src/" | dsh --profile headless -

# keluaran mesin (NDJSON) untuk diproses skrip
dsh --profile headless --json "jalankan test"

# melanjutkan sesi yang ada
dsh --profile headless --session-id session-… "lanjutkan"

# izin eksplisit untuk satu tugas
dsh --profile headless --permission-mode workspace-write \
    --add-dir /repo/.git "commit perbaikannya"
```

> 💡 **Untuk CI**
>
> Kombinasikan `--json` dengan `--approval-policy never` agar tindakan yang memerlukan persetujuan ditolak otomatis alih-alih menggantung menunggu manusia.

8.2 SDK Python

SDK Python membungkus perintah `dsh` yang sama beserta runtime native, sehingga eksekusi normal tidak memerlukan Node.js sistem. Prasyaratnya Python 3.10+, Git, dan platform Linux x64/arm64, macOS 14+ arm64, atau Windows x64.

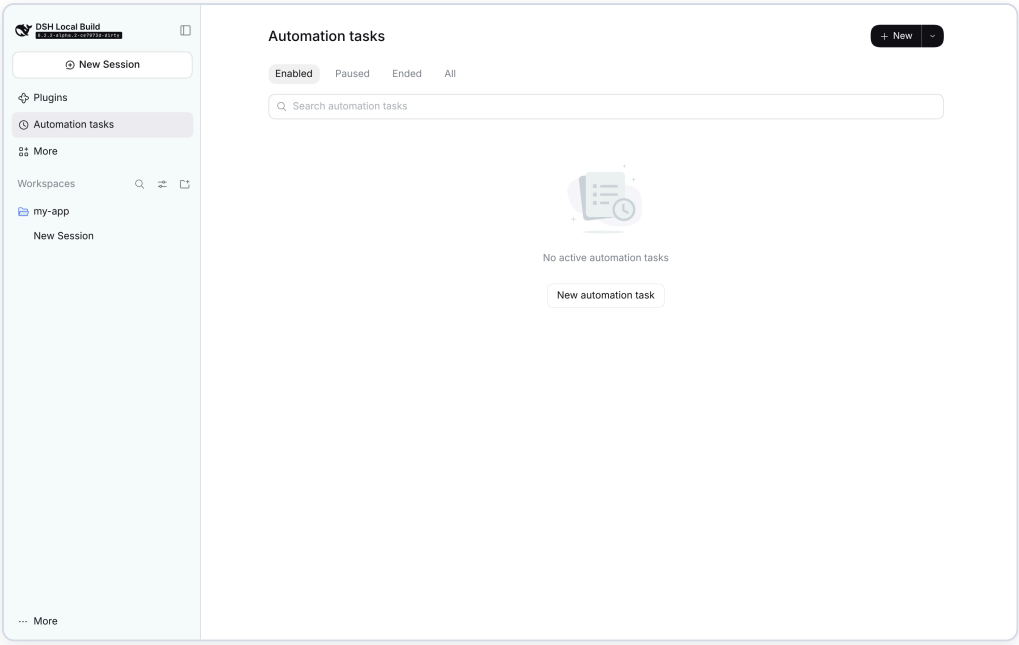
```bash
python -m venv .venv
. .venv/bin/activate
python -m pip install deepseek-harness-sdk
export DEEPSEEK_API_KEY=sk-...
python python/sdk/examples/minimal.py \
  --workspace /path/ke/workspace-sekali-pakai \
  --dsh-home /path/ke/dsh-home-contoh \
  --session-id example-001
```

Contoh di repositori menjalankan profil `sdk-minimal`. Gunakan workspace dan `DSH_HOME` yang terisolasi ketika bereksperimen.

8.3 Automation tasks: pengingat dan tugas berulang

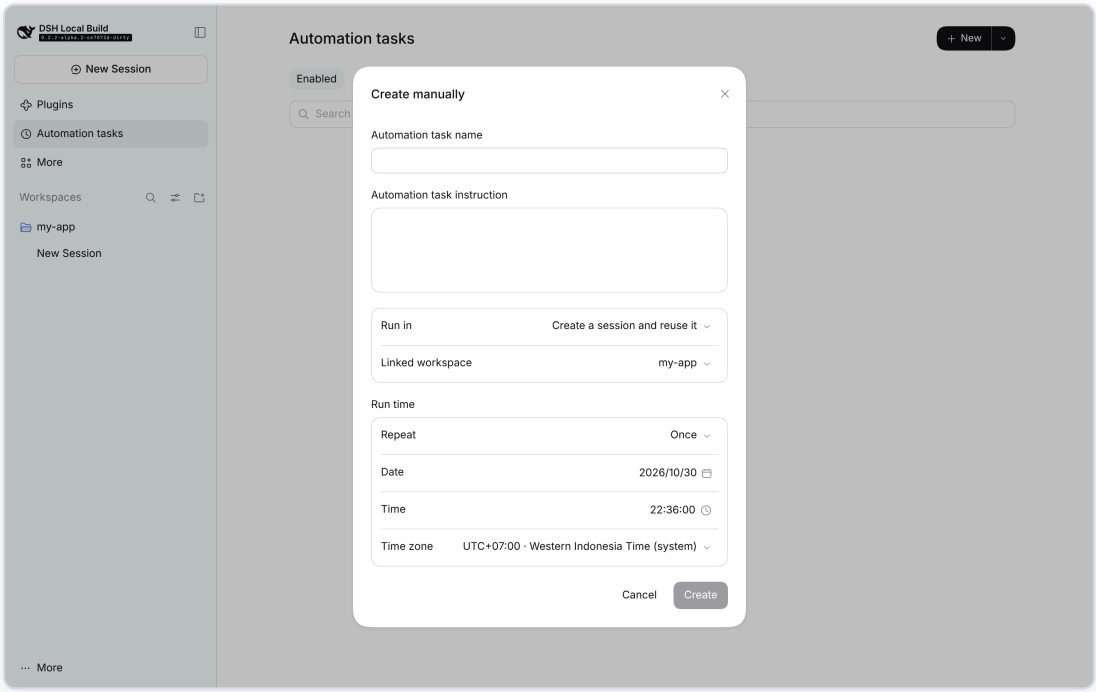
Profil Web menyertakan layanan Schedule. Cara termudah membuat tugas adalah memintanya dalam percakapan, misalnya: *“Ingatkan saya setiap hari pukul 23.00 WIB untuk memeriksa log deploy.”* Model memakai alat `schedule_create`, `schedule_list`, `schedule_view`, dan `schedule_edit`, lalu menampilkan kartu berisi nama dan frekuensi tugas.

Halaman **Automation tasks** di bilah samping menampilkan semua tugas dengan filter Enabled, Paused, Ended, dan All.



Gambar 8.2 Halaman Automation tasks yang masih kosong.

Untuk membuat tugas tanpa model, buka menu di samping **New** dan pilih **Create manually**.



Gambar 8.3 Dialog Create manually: nama, instruksi, tempat dijalankan, workspace, dan waktu.

- **Run in**: *Create a session and reuse it* (satu percakapan dipakai ulang) atau *New conversation for every run*.
- **Repeat**: Once, setiap N menit/jam, harian, mingguan, Senin–Jumat, bulanan, atau aturan kustom (RFC 5545).

- Detail tugas menyediakan **Pause**, **Resume**, **Run now**, **Edit**, dan **Delete task**, serta tab **Records** berisi riwayat pengiriman.

> **📌 Batasan penjadwalan**
>
> Tugas hanya dijadwalkan selama Host DSH berjalan. Saat dinyalakan ulang, DSH mengirim tugas sekali-jalan yang terlewat atau hanya kejadian terakhir yang terlewat dari tugas berulang. Pengiriman tepat-satu-kali tidak dijamin.

8.4 Review otomatis dari webhook GitHub

Sebagai overlay opsional, DSH dapat menerima webhook GitHub dan membuat sesi review hanya-baca ketika pull request berubah dari draft menjadi *ready for review*. Paket `@deepseek-ai/dsh-webhook` dan `@deepseek-ai/dsh-webhook-github` dipasang ke profil web, endpoint terpisah mendengarkan di `127.0.0.1:3081`, dan secret webhook disediakan lewat `DSH_GITHUB_WEBHOOK_SECRET`. Panduan lengkapnya ada di `docs/user/guide/github-review.md`.

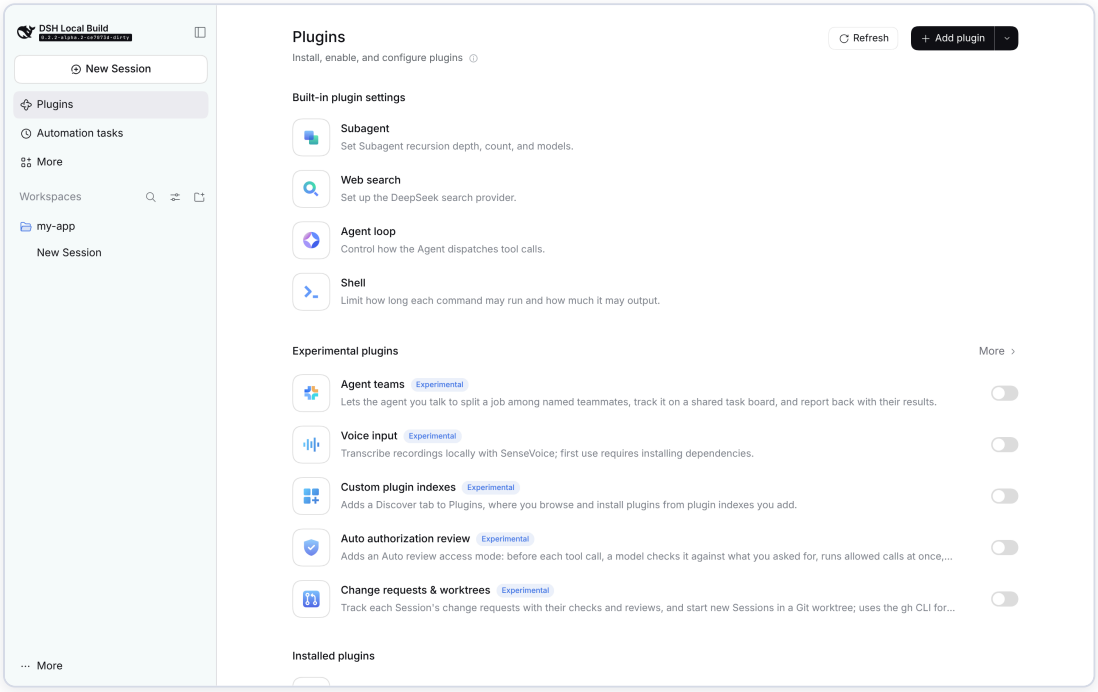
BAB 9

Plugin dan Kustomisasi

Karena semuanya plugin, hampir semuanya bisa diatur. Bab ini menunjukkan di mana tuas-tuasnya berada.

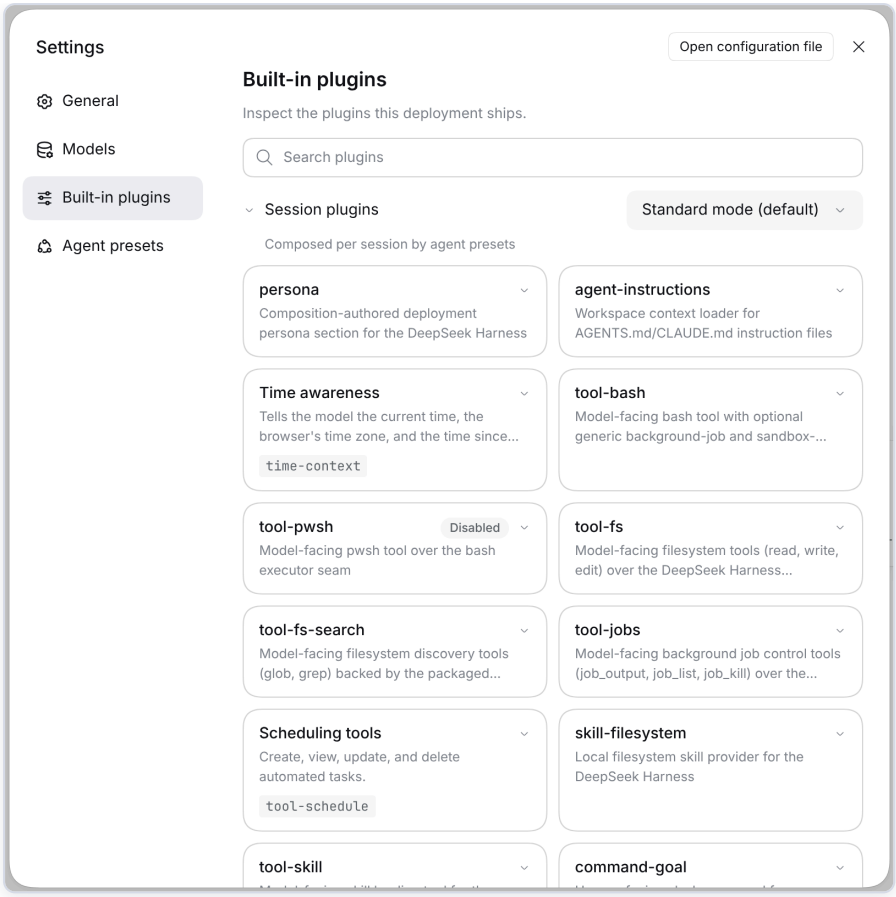
9.1 Halaman Plugins

Halaman **Plugins** mengelompokkan plugin menjadi tiga bagian:



Gambar 9.1 Halaman Plugins.

- **Built-in plugin settings** — pengaturan plugin inti seperti *Subagent* (kedalaman dan jumlah delegasi), *Web search*, *Agent loop*, dan *Shell* (batas waktu dan keluaran perintah).
- **Experimental plugins** — fitur yang sedang dievaluasi, misalnya *Agent teams*, *Voice input*, *Custom plugin indexes*, *Auto authorization review*, dan *Change requests & worktrees*. Aktifkan dengan sakelar di kanan.
- **Installed plugins** — plugin yang Anda pasang sendiri.



Gambar 9.2 Settings → Built-in plugins.

9.2 Memasang plugin lewat CLI

Plugin dipasang per profil. Perintah `dsh plugin` meneruskan argumen ke pnpm di direktori profil:

```bash
dsh plugin --profile web add @deepseek-ai/dsh-webhook @deepseek-ai/dsh-webhook-github
```

Instalasi dan startup profil memeriksa rentang `engines.dsh` plugin terhadap versi DSH yang berjalan; plugin yang tidak kompatibel memerlukan pengecualian versi yang diakui secara eksplisit.

9.3 Indeks plugin (Discover)

Aktifkan plugin eksperimental **Custom plugin indexes** untuk memunculkan tab **Discover** di halaman Plugins. DSH tidak menyertakan sumber indeks bawaan; tambahkan URL indeks melalui **Add index source**. Indeks adalah konten pihak ketiga — tambahkan hanya sumber yang Anda percayai dan periksa sumber paket pada dialog konfirmasi sebelum menekan **Install**.

9.4 Lapisan patch YAML

Setiap penyesuaian tingkat lanjut ditulis sebagai daftar *patch* di `cordis.patch.yml`: menimpa `config` sebuah entri berdasarkan `id`, menonaktifkan entri, atau menyisipkan entri baru dengan `insert`. Ekspresi JavaScript diperbolehkan dengan tag `!!js` di bawah `config`.

```yaml
# $DSH_HOME/profiles/web/cordis.patch.yml
- id: llm-deepseek
  config:
    reasoningEffort: max
```

> 💡 **Periksa sebelum menjalankan**
>
> Gunakan `dsh web --dump-config` untuk melihat hasil gabungan semua lapisan tanpa menjalankan server. Ini cara cepat memastikan patch Anda terbaca.

9.5 Creator mode: biarkan agen membuat plugin

Pilih **Creator mode** pada pemilih mode, atau **Let the agent create a plugin** dari menu Add plugin. Agen akan memakai preset `cordis` yang memahami struktur plugin DSH, lalu membantu Anda menulis plugin yang menambah fitur, UI, atau preset baru — vibe coding untuk alat vibe coding Anda sendiri.

BAB 10

Jaringan, Keamanan, dan Praktik Terbaik

Bab penutup ini merangkum hal-hal yang sering menjadi batu sandungan, sekaligus kebiasaan yang membuat vibe coding produktif untuk jangka panjang.

10.1 Bekerja di balik proxy

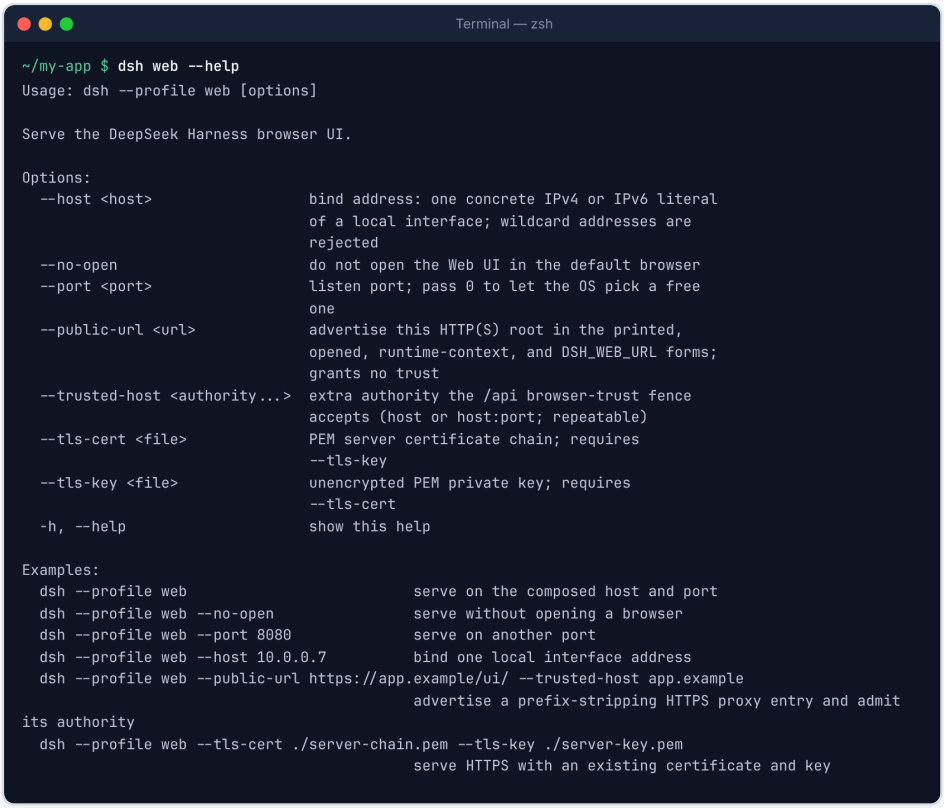
DSH merutekan permintaan keluar — panggilan model, pencarian web, pengambilan halaman, dan server MCP via HTTP — melalui variabel proxy standar yang dibaca saat peluncuran.

```bash
export HTTPS_PROXY=http://127.0.0.1:7890
export HTTP_PROXY=http://127.0.0.1:7890
export NO_PROXY=internal.example.com,.corp.example.com
```

- Simpan di profil shell atau di `$DSH_HOME/.env`. Berkas `.env` milik proyek **tidak boleh** mengatur proxy — DSH menolak berjalan agar repositori tidak bisa membelokkan lalu lintas Anda.
- DSH tidak membaca pengaturan proxy sistem operasi; ekspor variabelnya atau gunakan mode TUN.
- Proxy SOCKS tidak didukung; arahkan ke port HTTP aplikasi proxy Anda.
- `NO_PROXY` tidak menerima rentang CIDR; gunakan nama host atau sufiks domain.
- Proxy korporat yang menyadap TLS memerlukan `NODE_EXTRA_CA_CERTS=/path/ca.pem` sebelum `dsh` dijalankan.

10.2 Membuka Web UI di jaringan

Secara default server hanya mendengarkan di loopback. Opsi berikut tersedia saat Anda perlu membukanya lebih luas — sebaiknya di belakang reverse proxy berautentikasi.



Gambar 10.1 Opsi `dsh web --help`: host, port, URL publik, host tepercaya, dan TLS.

10.3 Daftar periksa keselamatan

1. Jalankan di VM, kontainer, atau akun pengguna terpisah untuk proyek yang tidak Anda percayai.
2. Pakai **Workspace Write** sebagai default; naikkan ke Full access hanya sementara dan secara sadar.
3. Commit pekerjaan Anda sebelum memulai tugas besar, agar mudah kembali dengan Git.
4. Baca setiap perintah di panel persetujuan sebelum menekan Allow once.
5. Jangan menempelkan kunci API ke dalam prompt; gunakan halaman Models atau variabel lingkungan.
6. Pasang plugin dan indeks plugin hanya dari sumber tepercaya.

10.4 Sepuluh kebiasaan vibe coder yang efektif

1. **Mulai dari definisi selesai.** Sertakan ‘test harus lulus’ atau kriteria terukur lain di setiap prompt.
2. **Rencanakan yang besar.** Gunakan `/plan` untuk perubahan lintas berkas.
3. **Satu sesi, satu tujuan.** Mulai sesi baru untuk topik yang berbeda agar konteks tetap bersih.
4. **Arahkan dengan @.** Sebut berkas yang relevan daripada berharap agen menebaknya.
5. **Tanamkan konvensi di AGENTS.md.** Tulis sekali, dipakai di setiap sesi.
6. **Tinjau diff, bukan hanya ringkasan.** Kartu Edit dan Write ada untuk dibaca.
7. **Iterasi kecil.** Minta satu perubahan per giliran; lebih mudah ditinjau dan dibatalkan.
8. **Padatkan saat perlu.** Gunakan Compact ketika percakapan menjadi sangat panjang.

9. **Simpan jejak.** Ekspor log sesi untuk dokumentasi atau pelaporan bug.
10. **Tetap penasaran.** Buka Trajectory ketika hasil terasa aneh — jawabannya biasanya ada di sana.

10.5 Ke mana selanjutnya?

- Dokumentasi resmi: `deepseek-harness.github.io/deepseek-harness`
- Panduan pengguna di repositori: `docs/user/guide/`
- Pengembangan plugin: `docs/user/develop/` dan `docs/cordis-primer.md`
- Katalog alat yang dilihat model: `docs/tool-catalog.md`
- Komunitas: GitHub Discussions dan Discord DeepSeek Harness

LAMPIRAN A

Lembar Contekan

Menjalankan	
`npx @deepseek-ai/dsh web`	Web UI tanpa instalasi permanen
`dsh web --no-open --port 8080`	Server tanpa membuka peramban, port lain
`dsh --profile headless "tugas"`	Satu tugas, cetak jawaban, keluar
`dsh --profile headless --json "tugas"`	Keluaran peristiwa NDJSON
`dsh web --patch ./extra.yml`	Menambah overlay konfigurasi
`dsh web --dump-config`	Melihat konfigurasi gabungan
`dsh plugin --profile web add <paket>`	Memasang plugin
Di composer	
`/plan <tugas>` · `/plan off`	Masuk / keluar plan mode
`/`	Daftar perintah: Compact, Permission, Model, Export, Goal, Feedback
`@nama`	Menyebut berkas, folder, atau sesi
Enter	Kirim pesan
Lokasi penting	
`~/.dsh` (`$DSH_HOME`)	Rumah DSH: profil, kredensial, skills
`~/.dsh/profiles/web/cordis.patch.yml`	Konfigurasi profil Web
`AGENTS.md`, `CLAUDE.md`	Instruksi proyek untuk agen
`.dsh/skills`, `.agents/skills`	Skills tingkat proyek

LAMPIRAN B

Glosarium

ACP
Agent Client Protocol; protokol stdio untuk klien otomasi.

Agent harness
Kerangka yang menghubungkan model bahasa dengan alat, izin, dan antarmuka sehingga dapat bertindak sebagai agen.

Bundle
Paket plugin yang menyumbang lapisan konfigurasi ke sebuah profil.

Composer
Kotak masukan tempat menulis prompt di Web UI.

Cordis
Kerangka kerja plugin yang menjadi fondasi DSH.

Headless
Mode tanpa antarmuka grafis: satu tugas, satu jawaban.

MCP
Model Context Protocol; standar untuk menghubungkan alat dan sumber data eksternal ke model.

Patch layer
Lapisan YAML (`cordis.patch.yml`) yang menimpa atau menambah konfigurasi plugin.

Plan mode
Mode ketika agen menyusun rencana untuk ditinjau sebelum eksekusi.

Preset (mode agen)
Komposisi alat dan instruksi agen: Standard, PTC, Minimal, Creator.

Profil
Tumpukan bundle plugin beserta lapisan konfigurasi pengguna di `$DSH_HOME/profiles`.

PTC
Mode yang memungkinkan agen memanggil banyak alat dalam satu program (`run_code`).

Sesi
Satu percakapan beserta log peristiwanya yang tersimpan.

Skill

Paket instruksi untuk jenis tugas tertentu yang dimuat sesuai kebutuhan.

Turn / giliran

Satu siklus dari pesan pengguna hingga agen selesai menjawab; berisi beberapa langkah.

Vibe coding

Gaya pemrograman dengan mendeskripsikan maksud dalam bahasa alami dan membiarkan AI menulis sebagian besar kode.

Workspace

Folder proyek yang terdaftar dan menjadi tempat kerja agen.

Daftar Pustaka

DeepSeek-AI. (2026). *DeepSeek Harness: Everything is a plugin* [Perangkat lunak]. GitHub. https://github.com/deepseek-ai/deepseek-harness

DeepSeek-AI. (2026). *DeepSeek Harness documentation*. https://deepseek-harness.github.io/deepseek-harness/

DeepSeek-AI. (2026). *Safety* (SAFETY.md) dan panduan pengguna (docs/user/guide) dalam repositori DeepSeek Harness, versi 0.2.2-alpha.2.

A programming paradigm for spatiotemporal composability. (2026). arXiv:2608.25512. https://arxiv.org/abs/2608.25512 [penulis belum terverifikasi]

Cordiverse. (t.t.). *Cordis* [Perangkat lunak]. GitHub. https://github.com/cordiverse/cordis

Biarkan AI mengetik.
Anda tetap memegang kemudi.

Vibe coding membuat ide berubah menjadi kode dalam hitungan menit — selama agen AI Anda bekerja dengan rencana yang jelas, izin yang tepat, dan hasil yang bisa diverifikasi. Buku ini memandu Anda menguasai DeepSeek Harness, agent harness sumber terbuka berarsitektur “semuanya adalah plugin”, melalui tutorial bertahap yang dilengkapi screenshot bernomor dari aplikasi aslinya.

Yang akan Anda pelajari:

- Memasang dan menjalankan DSH di macOS (Apple Silicon & Intel), Linux, dan Windows
- Menghubungkan DeepSeek, penyedia pihak ketiga, atau gateway sendiri
- Sesi coding pertama: to-do list, diff, test, dan iterasi
- Mengendalikan agen dengan mode, izin, persetujuan, dan plan mode
- Memberi konteks lewat AGENTS.md, skills, dan memori MCP
- Otomasi dengan mode headless, SDK Python, dan tugas terjadwal
- Kustomisasi plugin dan praktik keselamatan



[ISBN & barkod
diisi setelah terbit]

NONFIKSI / PEMROGRAMAN · KECERDASAN ARTIFISIAL